

AI-Based Real-Time Network Intrusion Detection System Using Machine Learning

Complete Project Learning & Technical Guide

MCA Mini Project — Complete Study & Reference Document

GitHub: github.com/vrajkiran/AI-NIDS

Technology Stack	Python 3.10+, Flask, Scapy, Scikit-learn, SQLite, Chart.js
ML Algorithm	Random Forest Binary Classifier (100 Trees)
Dataset	CICIDS2017 — Friday DDoS Traffic
Model Accuracy	99.97% (54,097 test samples)
Detection Mode	BENIGN / ATTACK (Binary Classification)
Frontend	Jinja2 Templates + Vanilla JavaScript + Chart.js

TABLE OF CONTENTS

Project Overview

8

1.1 Project Title

8

1.2 Simple One-Paragraph Explanation

8

1.3 System Flow at a Glance

8

1.4 Real-World Problem Solved

9

1.5 Aim

9

1.6 Objectives

9

1.7 Scope

9

1.8 Expected Outcome

10

Cybersecurity Basics

11

2.1 Computer Network

11

2.2 Internet vs LAN

11

2.3 IP Address

11

2.4 Port Number

11

2.5 Protocol

12

2.6 Network Packet

12

2.7 Network Traffic

12

2.8 Firewall vs IDS vs IPS

12

2.9 NIDS vs HIDS

13

2.10 Cyber Attack Types Relevant to This Project

13

Intrusion Detection Systems

14

3.1 What Is an IDS?

14

3.2 Two Classic IDS Approaches

14

3.3 Traditional IDS vs AI-based IDS

14

3.4 How AI-NIDS Works as a NIDS

14

Problem Statement

16

4.1 Increasing Network Threats

16

4.2 Limitations of Traditional Methods

16

4.3 The Core Problem This Project Addresses

16

4.4 Academic Scope Distinction	16
Proposed System Architecture	17
5.1 Overall Architecture	17
5.2 Component Descriptions	17
Complete Working Mechanism	19
6.1 Step-by-Step Process	19
6.2 Main Decision Flowchart	20
Network Packets — Deep Dive	22
7.1 What Is a Packet?	22
7.2 IP Packet Header Structure	22
7.3 What AI-NIDS Extracts from Each Packet	22
7.4 Why Payload Is Never Stored	23
Scapy — Packet Capture Engine	24
8.1 What Is Scapy?	24
8.2 How Scapy Captures Packets	24
8.3 The Scapy sniff() Call — Actual Code	24
8.4 Windows vs Linux Considerations	25
8.5 Fallback Mechanism	25
Network Flows	26
9.1 Packet vs Flow	26
9.2 The 5-Tuple Flow Key	26
9.3 Bidirectional Flow Matching	26
9.4 Flow State Tracked by AI-NIDS	27
Feature Extraction	28
10.1 Why Statistical Features?	28
10.2 Feature Formulas and Calculations	28
10.3 to_ml_features() — Actual Code	28
CICIDS2017 Dataset	30
11.1 What Is CICIDS2017?	30
11.2 Dataset File Used	30
11.3 Dataset Statistics	30
11.4 Dataset Processing Pipeline	30
11.5 Why CICIDS2017?	31

Data Preprocessing	32
12.1 Why Preprocessing Is Needed	32
12.2 Step-by-Step Preprocessing — Actual Implementation	32
12.3 MedianImputer Class — Actual Code	33
Machine Learning Basics	34
13.1 What Is Machine Learning?	34
13.2 Key Terminology	34
13.3 Training vs Prediction	34
Random Forest — The ML Algorithm	36
14.1 What Is a Decision Tree?	36
14.2 What Is Random Forest?	36
14.3 Why Random Forest for This Project?	36
14.4 Actual Training Configuration	37
Model Training	38
15.1 Training Command	38
15.2 Training Steps — Actual Code Flow	38
15.3 Saved Model Files	38
Model Evaluation	39
16.1 Evaluation Metrics Explained	39
16.2 Actual Results from evaluation_results.txt	39
16.3 Confusion Matrix	40
16.4 What Do These Numbers Mean Practically?	40
From Live Packet to ML Prediction	41
17.1 The Complete Inference Pipeline	41
17.2 Feature Order Matters	41
17.3 Known Limitation — Per-Packet vs Per-Flow Prediction	42
Flask Backend	43
18.1 What Is Flask?	43
18.2 Complete Route Reference	43
18.3 Thread Safety with monitoring_lock	43
18.4 read_model_metrics() — Parsing the Evaluation File	44
18.5 Flask Template Rendering	44

Database Design	45
19.1 Why SQLite?	45
19.2 Table: network_traffic	45
19.3 Table: alerts	45
19.4 Schema Auto-Migration	46
19.5 Key SQL Queries Used	46
Frontend Architecture	47
20.1 Technology Stack	47
20.2 Frontend-Backend Communication Flow	47
20.3 Jinja2 Templates Structure	47
20.4 Design Palette	48
20.5 JavaScript Polling — Actual Timing	48
Dashboard Components	49
21.1 Topbar Navigation	49
21.2 DEMO MODE ACTIVE Badge	49
21.3 Monitoring Control Panel	49
21.4 KPI Cards	49
21.5 Charts	50
Alert System	51
22.1 What Triggers an Alert?	51
22.2 Severity Assignment	51
22.3 Alert Status Flow	51
22.4 Alert Information Displayed	52
Real Monitoring Mode	53
23.1 Prerequisites	53
23.2 Step-by-Step Usage	53
23.3 What Happens in the Background	54
Demo Mode	55
24.1 Why Demo Mode Exists	55
24.2 Demo Data vs Real Data — Side-by-Side	55
24.3 The 10 Demo Traffic Records (Actual Data)	55
24.4 Clearing Demo Data	56
24.5 DEMO MODE ACTIVE Badge	56

One-Click Launcher	57
25.1 run_ai_nids.bat — Windows Batch Launcher	57
25.2 run.py — Python Launcher	57
25.3 Startup Sequence	58
Project File Structure	59
26.1 Directory Tree	59
Code Walkthrough	61
27.1 network/features.py — extract_packet_metadata()	61
27.2 network/flow.py — add_packet()	61
27.3 ml/predict.py — PredictionService.predict()	62
27.4 database/db.py — insert_network_traffic()	62
Complete End-to-End Example	64
28.1 Scenario: Your laptop browses https://www.google.com	64
Error Handling	66
Security & Privacy	67
30.1 Privacy Protections Built Into the Code	67
30.2 Disclaimer	67
Known Limitations	68
Future Improvements	69
Complete Installation Guide	70
33.1 Prerequisites	70
33.2 Installation Commands	70
33.3 Verify Installation	70
How To Use the Application	72
34.1 Launching the Application	72
34.2 Dashboard Controls Summary	72
34.3 For Project Demonstration	72
Troubleshooting	73
Project Development Journey	74
Quick Revision	76
37.1 One-Minute Explanation	76
37.2 Five-Minute Explanation	76

37.3 Complete Technical Flow Diagram 76

37.4 Glossary of Key Terms 77

CHAPTER 1

Project Overview

What is AI-NIDS and why does it exist?

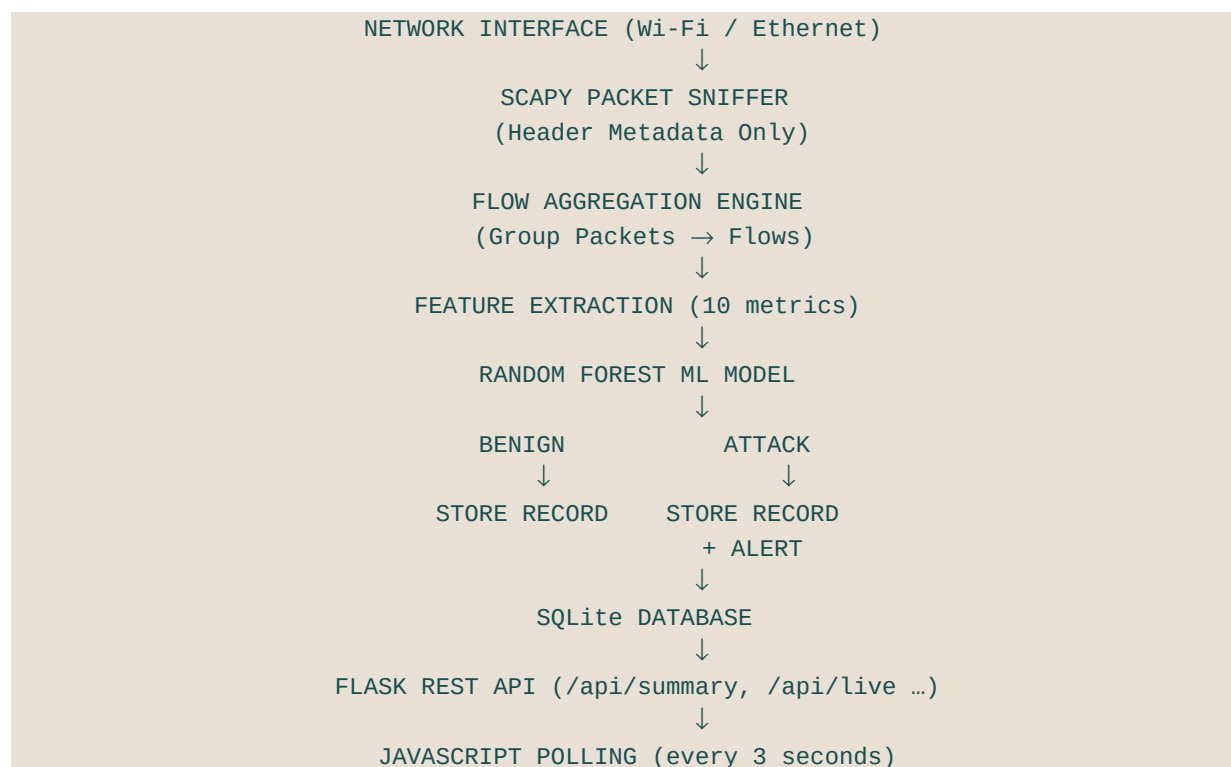
1.1 Project Title

AI-Based Real-Time Network Intrusion Detection System Using Machine Learning

1.2 Simple One-Paragraph Explanation

AI-NIDS is a software system that watches your computer's network connection in real time. It captures the header information of every packet (like letters on an envelope — who sent it, who receives it, and how large it is, but never the actual content). It groups these packets into network conversations called flows, calculates statistics about each flow, and feeds those statistics to a pre-trained Random Forest machine learning model. The model then decides whether each conversation is BENIGN (normal) or ATTACK (malicious). Results are stored in a local SQLite database and displayed live on a web dashboard at <http://127.0.0.1:5000>.

1.3 System Flow at a Glance



↓
WEB DASHBOARD (Charts, Tables, Alerts)

1.4 Real-World Problem Solved

Modern networks generate millions of packets per day. A human analyst cannot inspect each one individually. Traditional firewall rules match known patterns but fail against novel attacks. AI-NIDS uses a machine learning model trained on real attack data (CICIDS2017) to automatically classify every network conversation without manual rule updates.

1.5 Aim

To design, implement, and evaluate a stateful, passive Network Intrusion Detection System that classifies real-time network flows as BENIGN or ATTACK using a Random Forest classifier, without inspecting or storing raw packet payloads.

1.6 Objectives

- Passively capture IP packet headers from authorized network interfaces using Scapy.
- Aggregate packets into bidirectional 5-tuple network flows.
- Compute 10 statistical features from each flow matching the CICIDS2017 feature set.
- Classify each flow as BENIGN or ATTACK using a pre-trained Random Forest model.
- Store all flow results in SQLite, tagged as REAL or DEMO.
- Provide a live web dashboard with Chart.js visualizations.
- Support one-click launch via run_ai_nids.bat (Windows).
- Provide a DEMO MODE for presentation testing without live packet generation.

1.7 Scope

In Scope

- Passive monitoring of authorized local network adapters (Wi-Fi, Ethernet, Loopback).
- Binary classification: BENIGN vs ATTACK.
- SQLite persistence and Flask REST API.
- Web dashboard with live-updating charts and tables.
- Dual-mode operation: REAL capture and DEMO presentation mode.

Out of Scope

- Deep Packet Inspection (payload decryption or content analysis).
- Automatic firewall blocking or packet injection.
- Multi-node distributed sensor deployment.
- User authentication system.

1.8 Expected Outcome

A fully functional web dashboard accessible at <http://127.0.0.1:5000> that displays live network traffic records, detected attacks, severity alerts, and performance charts. The Random Forest model achieves 99.97% accuracy on the CICIDS2017 test set of 54,097 samples.

CHAPTER 2

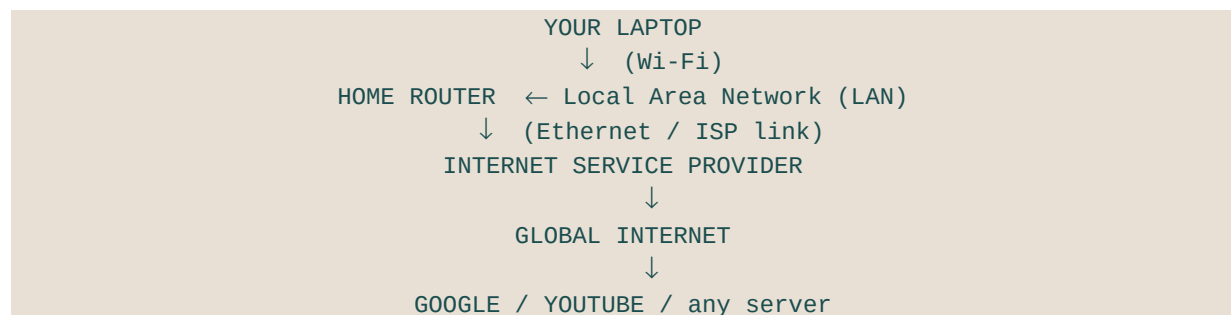
Cybersecurity Basics

Essential concepts before we understand the project

2.1 Computer Network

A **computer network** is a collection of computers and devices connected together to share resources and communicate. When your laptop connects to a router via Wi-Fi to access the internet, that is a network. Networks can be small (your home) or enormous (the global internet).

2.2 Internet vs LAN



A **LAN (Local Area Network)** is the private network inside your home or office. The **Internet** is the vast global network connecting millions of LANs worldwide.

2.3 IP Address

Every device on a network has an **IP address** — a numeric label like 192.168.1.105. Think of it as the postal address of your device. Two types exist: IPv4 (four numbers like 192.168.1.1) and IPv6 (longer, newer format). In this project, we work with IPv4 addresses extracted from packet headers.

2.4 Port Number

A **port** is a numbered channel on a device for a specific service. Your computer can run many services simultaneously:

Port Number	Protocol	Service
80	TCP	HTTP — Web browsing (unencrypted)
443	TCP	HTTPS — Web browsing (encrypted)
22	TCP	SSH — Secure remote login

53	UDP	DNS — Domain name resolution
8080	TCP	Alternative HTTP (often used by web apps)
445	TCP	SMB — Windows file sharing

[NOTE] Port 443 appearing in DEMO data (e.g., 192.168.1.105 → 104.21.52.12:443) means a standard HTTPS web browsing session — this is BENIGN.

2.5 Protocol

A **protocol** is a set of rules defining how data is formatted and transmitted. This project works with three key protocols:

Protocol	Full Name	Protocol Number	Characteristics
TCP	Transmission Control Protocol	6	Reliable, ordered delivery. Handshake required. Used for
UDP	User Datagram Protocol	17	Fast, no handshake. Used for DNS, streaming, VoIP.
ICMP	Internet Control Message Protocol	1	Control messages. Ping uses ICMP. No ports.

2.6 Network Packet

A **packet** is the fundamental unit of data transmission on a network. Large files are broken into many small packets, sent individually, and reassembled at the destination.

+-----+ PACKET HEADER (Metadata) Source IP: 192.168.1.105 Destination IP: 104.21.52.12 Source Port: 52410 Destination Port:443 Protocol: TCP (6) Packet Length: 1460 bytes Timestamp: 2026-08-30 20:50:12.342 +-----+ PAYLOAD (Not stored by AI-NIDS) Encrypted / raw data bytes +-----+
--

[ACTUAL PROJECT] AI-NIDS extracts ONLY the header fields shown above. The payload section is explicitly discarded. No passwords, no web content, no personal data is stored.

2.7 Network Traffic

Network traffic is the collective movement of packets across a network. When you watch YouTube, download a file, or send an email, your device generates network traffic. AI-NIDS monitors the metadata of this traffic to detect abnormal patterns.

2.8 Firewall vs IDS vs IPS

System	What It Does	Can Block?	AI-NIDS Type
Firewall	Filters packets by fixed rules (IP/port)	Yes	No
IDS (Intrusion Detection System)	Detects suspicious activity, raises alerts	No (detect only)	Yes — AI-NIDS is an IDS
IPS (Intrusion Prevention System)	Detects AND actively blocks threats	Yes	No — AI-NIDS is passive

2.9 NIDS vs HIDS

An IDS can monitor at two levels:

- **NIDS (Network IDS)** — Monitors network traffic flowing through a network segment. AI-NIDS is a NIDS. It inspects packets at the network level.
- **HIDS (Host IDS)** — Monitors activities on a single computer (file changes, log files, processes). AI-NIDS does NOT implement HIDS functionality.

2.10 Cyber Attack Types Relevant to This Project

Attack Type	Description	Relevance to AI-NIDS
DDoS	Distributed Denial of Service — overwhelming a server with requests	AI-NIDS 2017 dataset contains DDoS flows. Model trained on these can detect.
Port Scanning	Systematically probing ports to find open services	Generates high packet rate flows. Model may detect.
Brute Force	Repeated login attempts	Creates many small flows to port 22 (SSH) or 80 (HTTP).
Data Exfiltration	Unauthorized large data transfer outward	Shows as high byte count flows.

CHAPTER 3

Intrusion Detection Systems

Understanding IDS, NIDS, and why ML helps

3.1 What Is an IDS?

An **Intrusion Detection System (IDS)** is a security monitoring tool that inspects network traffic or system activity and generates alerts when it detects suspicious or potentially malicious behavior. Think of it as a security camera system for your network — it watches everything and calls for attention when something unusual happens.

3.2 Two Classic IDS Approaches

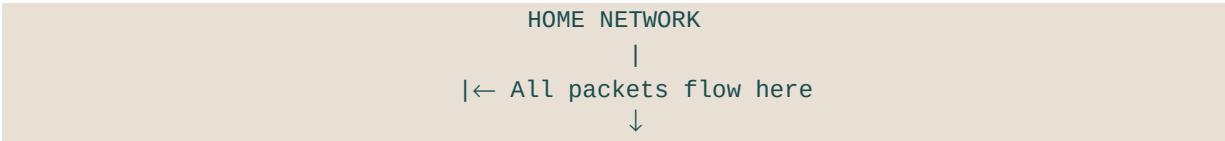
Approach	How It Works	Advantage	Disadvantage
Signature-Based	Matches traffic against a database of known attack signatures (like Snort)	Can detect known attacks	Cannot detect new (zero-day) attacks. Needs updates
Anomaly-Based	Learns 'normal' behavior and alerts when deviations occur	Can detect unknown attacks	Higher false positive rate. Requires training data

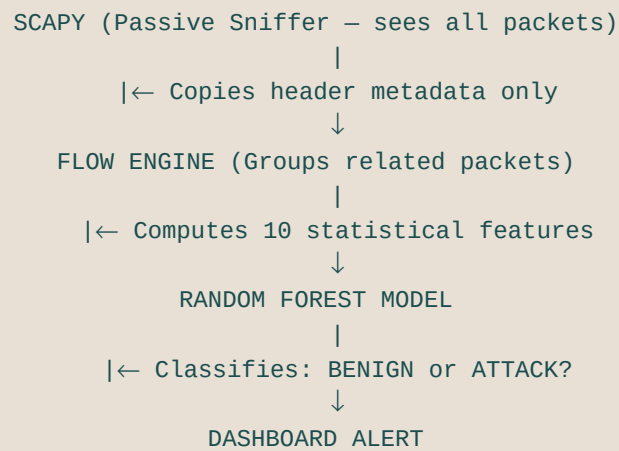
AI-NIDS uses a **supervised ML approach**, which is closer to anomaly-based but trained on labeled examples of both BENIGN and ATTACK traffic.

3.3 Traditional IDS vs AI-based IDS

Property	Traditional Signature IDS	AI-NIDS (This Project)
Detection Method	Fixed rule matching	Random Forest ML model
Zero-Day Attack Detection	No	Possible (if similar to training data)
Update Required	Manual signature updates	Retrain model on new data
Feature Used	Packet content bytes	Flow-level statistical metadata
Privacy Risk	May inspect payload content	Metadata only — no payload stored
Speed	Very fast	Fast — model inference in milliseconds
Accuracy (CICIDS2017)	N/A	99.97%

3.4 How AI-NIDS Works as a NIDS





[NOTE] Passive sniffing means Scapy reads packet copies that the OS network stack delivers. It does NOT intercept or modify traffic. Normal communications are not disrupted.

CHAPTER 4

Problem Statement

The specific problem this project solves

4.1 Increasing Network Threats

Network attacks are growing in frequency and sophistication. DDoS attacks have reached terabit-per-second scales. Attackers use encrypted traffic, randomized timing, and distributed botnets to evade traditional detection. Manual inspection by security analysts is impossible at modern network speeds.

4.2 Limitations of Traditional Methods

- **Static Firewall Rules** — Block known bad IPs but miss attack patterns from new sources.
- **Signature IDS** — Requires manual rule updates; blind to novel attack variants.
- **Manual Analysis** — A human cannot inspect millions of packets per day.
- **Payload Inspection** — Decrypting HTTPS traffic violates privacy and requires expensive hardware.

4.3 The Core Problem This Project Addresses

How can we automatically detect malicious network activity in real time, using only non-invasive packet header metadata, without storing private user data, with high accuracy, and with a visual dashboard for monitoring?

4.4 Academic Scope Distinction

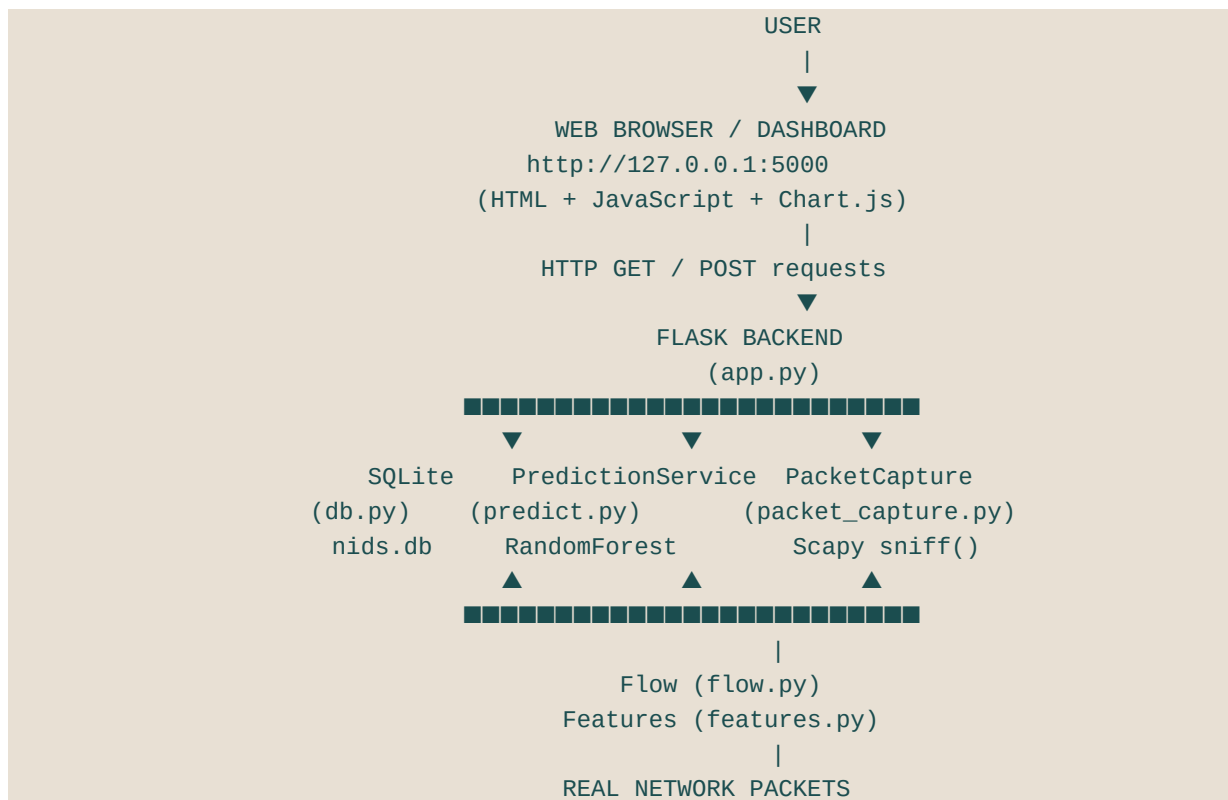
[IMPORTANT] This is an MCA Mini Project for academic demonstration. It monitors a single local machine's network interface. It does NOT replace enterprise-grade IDS tools like Snort or Suricata. It is designed to demonstrate the ML-based detection concept clearly and correctly.

CHAPTER 5

Proposed System Architecture

How the solution is designed

5.1 Overall Architecture



5.2 Component Descriptions

Web Dashboard (templates/ + static/)

Five HTML pages rendered by Flask/Jinja2: Dashboard, Live Traffic, Alerts, Model Performance. Vanilla JavaScript polls five REST API endpoints every 3 seconds and updates Chart.js charts and HTML table rows dynamically without full page reloads.

Flask Backend (app.py)

The central coordinator. Defines all HTTP routes, manages the monitoring thread lifecycle with `threading.Lock`, reads model metrics from `evaluation_results.txt`, and serves JSON data to the frontend.

SQLite Database (database/db.py + nids.db)

Two tables: network_traffic (stores each classified flow) and alerts (stores ATTACK events). Each record has a data_source column: 'REAL' for live Scapy captures, 'DEMO' for seeded presentation data.

PredictionService (ml/predict.py)

Loads three Joblib binary files on startup: the trained RandomForestClassifier model, the ordered feature list, and the MedianImputer preprocessing object. Provides the predict() method that returns BENIGN/ATTACK, confidence, and severity.

PacketCapture (network/packet_capture.py)

Runs Scapy sniff() in a background daemon thread. For each IP packet received, calls extract_packet_metadata() (features.py), updates flow state (_update_flow), calls PredictionService.predict() with flow features, and triggers save_detection() in app.py to persist the result.

Flow Engine (network/flow.py)

The Flow class tracks one bidirectional network conversation. It maintains separate forward and backward packet/byte counters and timestamps. Its to_ml_features() method returns a dictionary of the 10 features expected by the model.

CHAPTER 6

Complete Working Mechanism

Step-by-step walkthrough from user click to dashboard update

6.1 Step-by-Step Process

STEP 1 — User Opens Application

User double-clicks `run_ai_nids.bat`. This runs `run.py` which calls `create_database()` to initialize `nids.db`, starts Flask on `127.0.0.1:5000`, waits 1.5 seconds, then automatically opens the default browser to the dashboard URL.

STEP 2 — Dashboard Loads

Flask renders `dashboard.html` via the `/` route, passing summary stats, recent traffic rows, recent alerts, monitoring status, list of Scapy-detected interfaces, and `default_duration` from config.

STEP 3 — JavaScript Begins Polling

`main.js` calls `refreshPage()` immediately, then sets a 3-second interval. `refreshPage()` runs 6 `async fetch()` calls in parallel: `/api/status`, `/api/summary`, `/api/charts`, `/api/live`, `/api/alerts`, and dashboard table refreshes.

STEP 4 — User Selects Interface and Starts Monitoring

User selects a network interface (e.g., 'Wi-Fi') from the dropdown, sets capture duration (default: 10 seconds from `config.py` `CAPTURE_DURATION`), and clicks **START MONITORING**. A POST request goes to `/start-monitoring`.

STEP 5 — Flask Validates and Starts Background Thread

Flask validates the interface name exists in `get_interface_names()` and duration is between 5 and 3600 seconds. It creates a `PacketCapture` object with `enabled=True` and `prediction_enabled=True`, starts a daemon thread running `run_monitor()`, and redirects to dashboard.

STEP 6 — Scapy Begins Packet Sniffing

`PacketCapture.start()` calls `load_predictor()` (loads the Random Forest model), then calls `scapy.sniff(iface=interface, prn=handle_packet, timeout=duration, store=False, stop_filter=lambda _: stop_event.is_set())`. `store=False` means Scapy does NOT keep packets in memory.

STEP 7 — Each Packet is Processed

For each IP packet received by Scapy, `handle_packet()` is called. `extract_packet_metadata()` extracts: timestamp, source IP, destination IP, source port, destination port, protocol number, packet length. Non-IP packets return None.

STEP 8 — Packet is Added to a Flow

`_update_flow()` checks if a matching flow already exists using the 5-tuple key (`src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`) or its reverse. If no match, a new Flow object is created. Packet stats are added to the flow counters.

STEP 9 — Flow Features are Extracted

`flow.to_ml_features()` computes the 10 features: Destination Port, Protocol, Flow Duration (microseconds), Total Fwd Packets, Total Backward Packets, Total Length of Fwd Packets, Total Length of Bwd Packets, Fwd Packet Length Mean, Bwd Packet Length Mean, Flow Bytes/s, Flow Packets/s.

STEP 10 — Random Forest Makes a Prediction

`PredictionService.predict(features)` validates all 10 feature values are numeric, builds a one-row DataFrame in the correct column order, applies the MedianImputer (for any NaN values), calls `model.predict()` to get 0/1, and `model.predict_proba()` to get confidence probability.

STEP 11 — Prediction is Classified

`prediction_number=0` → 'BENIGN'. `prediction_number=1` → 'ATTACK'. If confidence ≥ 0.90 and ATTACK: severity = 'HIGH'. If confidence < 0.90 and ATTACK: severity = 'MEDIUM'. BENIGN: 'No alert'.

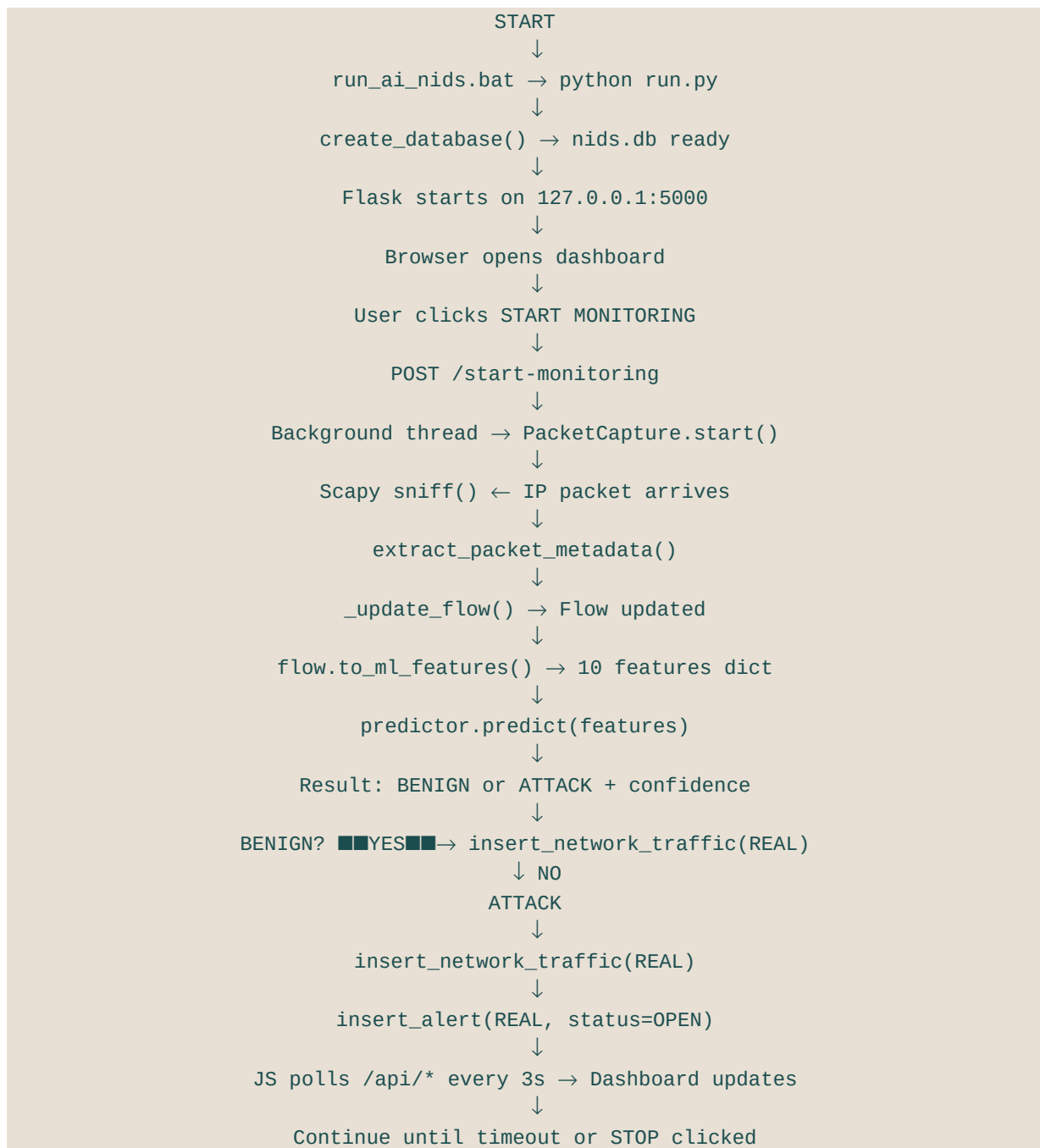
STEP 12 — Result Saved to SQLite

`save_detection()` in `app.py` formats a timestamp, calls `insert_network_traffic()` with `data_source='REAL'` to save the flow record. If `prediction=='ATTACK'`, also calls `insert_alert()` with `data_source='REAL'`, `status='OPEN'`.

STEP 13 — Dashboard Automatically Updates

JavaScript's 3-second interval fires. `/api/summary` returns updated counts. `/api/live` returns the latest 50 traffic rows. `/api/charts` returns aggregated chart data. JavaScript updates KPI card numbers, rebuilds table HTML, and updates Chart.js datasets.

6.2 Main Decision Flowchart



CHAPTER 7

Network Packets — Deep Dive

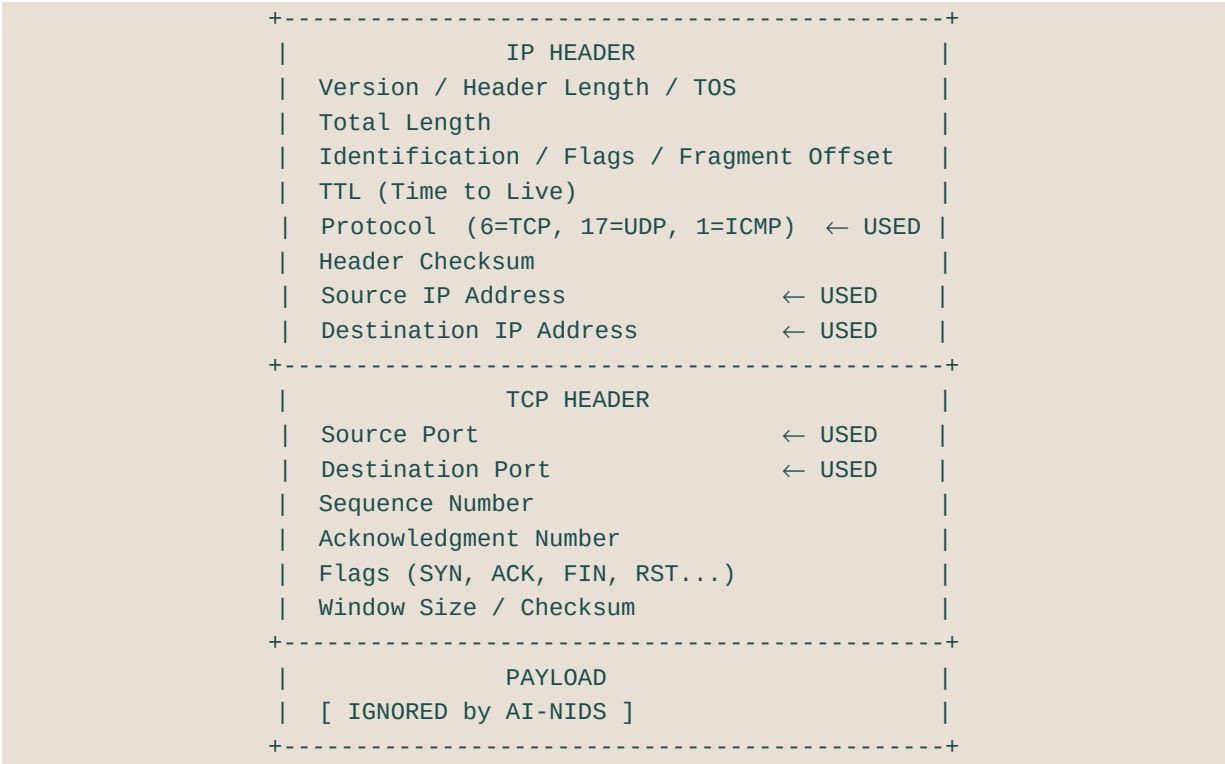
What a packet is and how AI-NIDS reads it

7.1 What Is a Packet?

When your computer sends data over a network (e.g., loading a web page), the data is broken into small chunks called **packets**. Each packet can travel independently and may take different paths to the destination, where they are reassembled in order.

Every packet has two parts: a **header** (metadata describing the packet) and a **payload** (the actual data content). AI-NIDS uses ONLY the header.

7.2 IP Packet Header Structure



7.3 What AI-NIDS Extracts from Each Packet

The function `extract_packet_metadata()` in `network/features.py` extracts:

Field Extracted	Source	Example Value	Purpose in AI-NIDS
-----------------	--------	---------------	--------------------

timestamp	packet.time (Unix epoch)	1756567212.342	Flow duration calculation
source_ip	packet[IP].src	192.168.1.105	Flow identification
destination_ip	packet[IP].dst	104.21.52.12	Flow identification
source_port	packet[TCP/UDP].sport	52410	Flow key
destination_port	packet[TCP/UDP].dport	443	ML feature: Destination Port
protocol	packet[IP].proto	6 (TCP)	ML feature: Protocol
packet_length	len(packet)	1460	Byte counting for features
protocol_name	Lookup dict {1:'ICMP',6:'TCP',17:'UDP'}	TCP	Database display

7.4 Why Payload Is Never Stored

- **Privacy:** Payloads may contain passwords, personal messages, or bank details.
- **Legal protection:** Capturing payload data without authorization is illegal in most countries.
- **Unnecessary for ML:** Flow statistics (rates, counts, sizes) are sufficient to classify attacks.
- **Storage efficiency:** Headers are tiny (40-60 bytes) vs payloads (up to 65,535 bytes).

CHAPTER 8

Scapy — Packet Capture Engine

How AI-NIDS captures packets from the network

8.1 What Is Scapy?

Scapy is a powerful Python library for packet manipulation, sniffing, crafting, and dissection. In AI-NIDS, it is used only for **passive sniffing** — reading copies of packets from the network interface without modifying or injecting anything.

8.2 How Scapy Captures Packets

Scapy calls the operating system's network capture API (Npcap on Windows, libpcap on Linux/macOS). The network card is placed in **promiscuous mode**, meaning it receives all packets on the network segment, not just those addressed to your machine.

```

Network Interface Card (Wi-Fi / Ethernet)
      |
      Npcap / libpcap driver
      |
      Scapy sniff()
      |
      prn=handle_packet callback
      |
      Called for each arriving packet
  
```

8.3 The Scapy sniff() Call — Actual Code

```

sniff(
    iface=self.interface or None,    # Network adapter to listen on
    prn=self.handle_packet,          # Function called per packet
    timeout=self.duration,           # Stop after N seconds
    store=False,                     # Don't keep raw packets in memory
    stop_filter=lambda _: self.stop_event.is_set() # Stop if user clicks STOP
)
  
```

Line-by-line explanation:

- **iface**: Which network adapter to sniff. If empty, Scapy picks its default. The user selects from the dropdown which shows all adapters detected by `get_interface_names()`.
- **prn**: The callback function. Scapy calls `handle_packet(packet)` for every received packet.
- **timeout**: Automatically stops sniffing after this many seconds (user-configured, 5–3600).
- **store=False**: Critical privacy setting. Tells Scapy NOT to accumulate packets in RAM. Each packet is processed and discarded.

- **stop_filter**: Checked after each packet. Returns True when the user clicks ■ STOP, signalling Scapy to exit the sniff loop.

8.4 Windows vs Linux Considerations

Platform	Driver Required	Notes
Windows	Npcap (installed separately)	WinPcap API-compatible mode required for Scapy. AI-NIDS installed Npcap with
Linux/macOS	libpcap (usually pre-installed)	May require running as root (sudo) for raw socket access.

8.5 Fallback Mechanism

If Scapy cannot access Layer-2 (raw Ethernet frames) because Npcap is missing or misconfigured, packet_capture.py automatically falls back to Layer-3 IP socket sniffing:

```
except RuntimeError as error:
    if 'winpcap is not installed' not in str(error).lower():
        # Fatal error – stop
        self.error_message = 'Scapy capture error: ' + str(error)
        return
    # Try Layer-3 fallback
    socket = conf.L3socket(iface=self.interface or None)
    sniff(opened_socket=socket, prn=self.handle_packet,
          timeout=self.duration, store=False,
          stop_filter=lambda _: self.stop_event.is_set())
```

[ACTUAL PROJECT] After installing Npcap (which we did), conf.use_pcap = True and the fallback is not needed. Full Layer-2 packet capture is now active.

CHAPTER 9

Network Flows

Why packets are grouped into flows before ML classification

9.1 Packet vs Flow

A single network **packet** carries very little information on its own. A single packet might be 60 bytes from IP 10.0.0.1 to 8.8.8.8 on port 443. Is that normal or suspicious? Impossible to tell from one packet alone.

A **flow** is a group of packets belonging to the same network conversation. By grouping packets and computing statistics (how many? how fast? how many bytes?), we get meaningful features that can reveal attack patterns.

```
PACKET 1 (src:192.168.1.105:52410 → dst:104.21.52.12:443, TCP, 60 bytes) ■■
PACKET 2 (src:104.21.52.12:443 → dst:192.168.1.105:52410, TCP, 1460 bytes) ■■
PACKET 3 (src:192.168.1.105:52410 → dst:104.21.52.12:443, TCP, 200 bytes) ■■→ FLOW
PACKET 4 (src:104.21.52.12:443 → dst:192.168.1.105:52410, TCP, 1460 bytes) ■■

All 4 packets share the same 5-tuple → they belong to one HTTPS flow.
```

9.2 The 5-Tuple Flow Key

A flow is uniquely identified by five fields — the **5-tuple**:

Field	Example Value	Purpose
Source IP	192.168.1.105	Originating host
Destination IP	104.21.52.12	Target host
Source Port	52410	Originating service/ephemeral port
Destination Port	443	Target service port
Protocol	6 (TCP)	Transport protocol

9.3 Bidirectional Flow Matching

Network conversations are bidirectional. When computer A sends to B, B also replies back. AI-NIDS groups both directions into one flow using forward and reverse key matching:

```
def _update_flow(self, metadata: dict):
    flow_key = metadata_to_flow_key(metadata) # (srcIP, dstIP, srcPort, dstPort, proto)
    reverse_key = metadata_to_reverse_flow_key(metadata) # (dstIP, srcIP, dstPort, s
```

```
rcPort, proto)

    if flow_key in self.flows:          # Forward packet – flow exists
        flow = self.flows[flow_key]
    elif reverse_key in self.flows:    # Reverse packet – same flow, opposite direction
        flow = self.flows[reverse_key]
    else:                             # New conversation – create flow
        flow = Flow(src_ip, dst_ip, src_port, dst_port, protocol)
        self.flows[flow_key] = flow
```

9.4 Flow State Tracked by AI-NIDS

The Flow class (network/flow.py) tracks:

Attribute	Description
start_time	Timestamp of the first packet in this flow
last_time	Timestamp of the most recent packet
forward_packet_count	Number of packets from source → destination
backward_packet_count	Number of packets from destination → source
forward_byte_count	Total bytes in forward direction
backward_byte_count	Total bytes in backward direction

CHAPTER 10

Feature Extraction

The 10 statistical features used by the ML model

10.1 Why Statistical Features?

The Random Forest model cannot directly understand raw packets. It needs numeric features. Statistical flow features capture the **behavior** of a conversation: How fast? How much data? How many packets? These patterns differ sharply between normal browsing and a DDoS flood.

10.2 Feature Formulas and Calculations

[ACTUAL PROJECT] These are the ACTUAL 10 features from SELECTED_FEATURES in ml/preprocess.py and computed by flow.to_ml_features() in network/flow.py.

#	Feature Name	Formula / Source	CICIDS2017	Give Scapy?	Why Useful for Attack Detection
1	Destination Port	packet[TCP/UDP].dport	Yes	Yes	High-risk ports (22, 445, 8080) are common attack targets.
2	Flow Duration	(last_time - start_time) * 1,000,000	Yes	Yes	DDoS floods have very short duration per flow. Long SSH sessions are normal.
3	Total Fwd Packets	count of forward packets	Yes	Yes	DDoS has enormous forward packet counts. Scanning has many small flows.
4	Total Backward Packets	count of reverse packets	Yes	Yes	Asymmetric flows (many fwd, few bwd) indicate one-way flood.
5	Total Length of Fwd Packets	Σ forward packet lengths (bytes)	Yes	Yes	DDoS uses large forward byte volumes.
6	Total Length of Bwd Packets	Σ backward packet lengths (bytes)	Yes	Yes	Low backward bytes with high forward bytes indicates flood.
7	Fwd Packet Length Mean	Fwd Bytes / Fwd Packets	Yes	Yes	DDoS often sends fixed-size packets. Normal traffic varies in size.
8	Bwd Packet Length Mean	Bwd Bytes / Bwd Packets	Yes	Yes	Server response size reveals service type and abnormal behavior.
9	Flow Bytes/s	Total Bytes / Duration (seconds)	Yes	Yes	DDoS floods have extremely high bytes/sec. Normal HTTPS is ~100KB/s.
10	Flow Packets/s	Total Packets / Duration (seconds)	Yes	Yes	DDoS: thousands of packets/sec. Normal DNS: 1-2 packets/sec.

10.3 to_ml_features() — Actual Code

```
def to_ml_features(self) -> dict:
    duration_seconds = self.duration_seconds          # last_time - start_time
    duration_microseconds = duration_seconds * 1_000_000
    bytes_per_second = self.total_bytes / duration_seconds if duration_seconds > 0 else 0
    packets_per_second = self.total_packets / duration_seconds if duration_seconds > 0 else 0

    return {
```

```
'Destination Port':      self.destination.port,
'Protocol':              self.protocol,
'Flow Duration':         duration_microseconds,
'Total Fwd Packets':     self.forward_packet_count,
'Total Backward Packets': self.backward_packet_count,
'Total Length of Fwd Packets': self.forward_byte_count,
'Total Length of Bwd Packets': self.backward_byte_count,
'Fwd Packet Length Mean': fwd_bytes/fwd_pkts if fwd_pkts else 0,
'Bwd Packet Length Mean': bwd_bytes/bwd_pkts if bwd_pkts else 0,
'Flow Bytes/s':         bytes_per_second,
'Flow Packets/s':       packets_per_second,
}
```

[NOTE] IMPORTANT: The feature list in `ml/preprocess.py` has 10 entries but `flow.to_ml_features()` returns 11 key-value pairs (includes 'Protocol' which is in `flow.py` but NOT in **SELECTED FEATURES**). `PredictionService.arrange_features()` orders the `DataFrame` using `self.feature_list` loaded from `feature_list.joblib` — so only the 10 trained features are used for prediction.

CHAPTER 11

CICIDS2017 Dataset

The training data that taught the model what attacks look like

11.1 What Is CICIDS2017?

CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset 2017) is a benchmark dataset created by the University of New Brunswick, Canada. It contains labeled network flow records generated in a controlled lab environment where both normal and attack traffic was generated systematically.

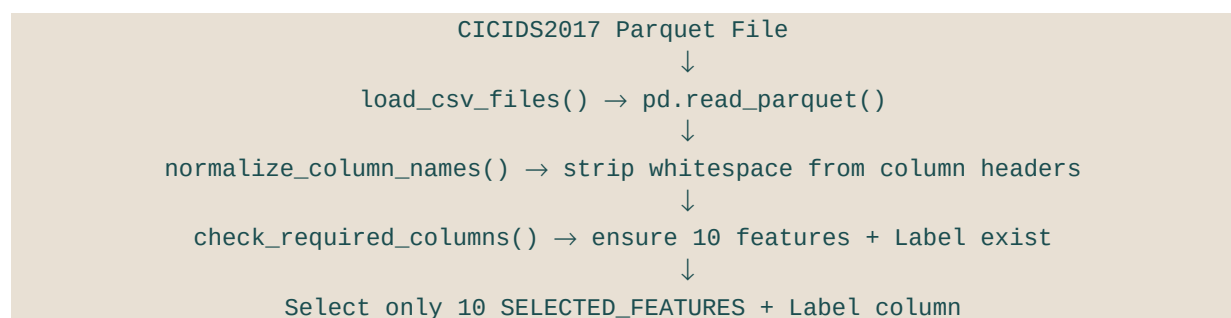
11.2 Dataset File Used

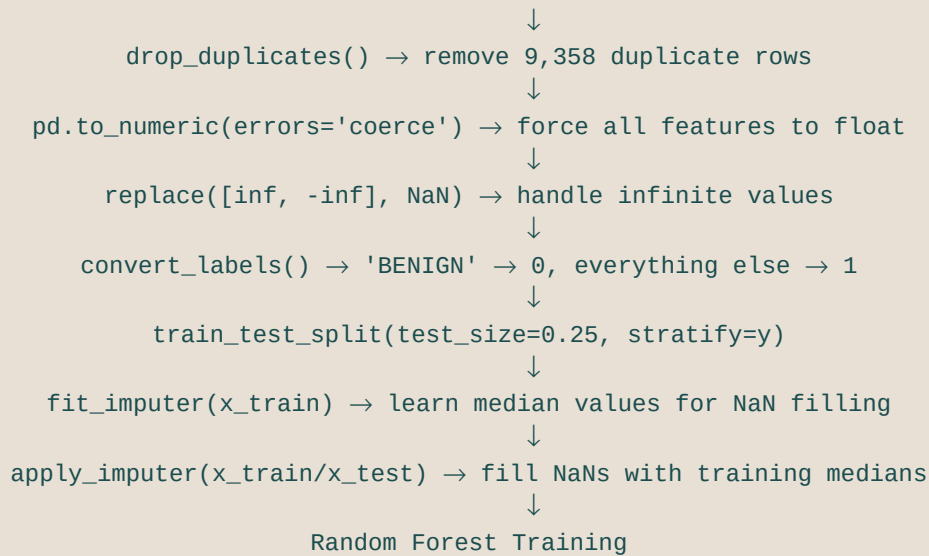
[ACTUAL PROJECT] File: data/Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv.parquet — Friday afternoon DDoS attack traffic and benign background traffic.

11.3 Dataset Statistics

Metric	Value
Total rows loaded	225,745
Duplicate rows removed	9,358
Cleaned dataset size	216,387 rows
BENIGN flows	89,199 (41.2%)
ATTACK flows (DDoS)	127,188 (58.8%)
Training set (75%)	162,290 rows
Test set (25%)	54,097 rows

11.4 Dataset Processing Pipeline





11.5 Why CICIDS2017?

- Publicly available academic benchmark — used in hundreds of research papers.
- Contains labeled flows with both BENIGN and known attack categories.
- Features are pre-computed flow statistics (matching what AI-NIDS calculates live).
- The Friday afternoon file contains high-volume DDoS traffic — ideal for binary classification demo.

CHAPTER 12

Data Preprocessing

How raw CICIDS2017 data is cleaned before training

12.1 Why Preprocessing Is Needed

Real-world datasets are messy. CICIDS2017 has column names with extra spaces, some values that are infinite or missing (NaN), and duplicate rows. ML models need clean, numeric, consistent data to train correctly.

12.2 Step-by-Step Preprocessing — Actual Implementation

Step 1 — Column Name Normalization

`normalize_column_names()` calls `dataframe.columns.str.strip()`. CICIDS2017 CSV files sometimes have column headers with leading/trailing spaces like ' Destination Port' instead of 'Destination Port'. Stripping removes these so feature selection works correctly.

Example: ' Destination Port ' → 'Destination Port'

Step 2 — Select Only Required Columns

Only the 10 `SELECTED_FEATURES` and the 'Label' column are kept. The full CICIDS2017 dataset has 80+ columns. Keeping only 11 reduces memory and speeds up training significantly.

Example: 80 columns → 11 columns

Step 3 — Remove Duplicate Rows

`dataframe.drop_duplicates()` removes 9,358 rows that are exact copies. Duplicates can bias the model to overfit on repeated patterns.

Example: 225,745 → 216,387 rows

Step 4 — Convert to Numeric

`pd.to_numeric(column, errors='coerce')` converts every feature column to float. If a value cannot be converted (e.g., 'Infinity', 'N/A' text), it becomes NaN. This handles corrupt or non-numeric entries safely.

Example: 'Infinity' string → NaN float

Step 5 — Handle Infinite Values

`dataframe.replace([np.inf, -np.inf], np.nan)` replaces positive and negative infinity with NaN. Flow Bytes/s can produce Inf if duration is zero. These are then handled by the imputer.

Example: inf → NaN → median value

Step 6 — Binary Label Conversion

`convert_labels()` converts the Label column: 'BENIGN' → 0, everything else → 1. In the Friday DDoS file, non-BENIGN labels include 'DDoS'. Using `np.where` makes this efficient.

Example: 'BENIGN' → 0, 'DDoS' → 1

Step 7 — Train/Test Split

`train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)`. `stratify=y` ensures the same BENIGN:ATTACK ratio in both train and test sets. 75% (162,290) for training, 25% (54,097) for testing.

Example: 162,290 train + 54,097 test

Step 8 — Median Imputation

`MedianImputer.fit(x_train)` computes the median of each feature column from training data. `MedianImputer.transform(dataframe)` replaces any remaining NaN values with the learned medians. The imputer is saved to `preprocessing_info.joblib` so the same medians are used for live traffic during prediction.

Example: NaN → median of training feature column

12.3 MedianImputer Class — Actual Code

```
class MedianImputer:
    def fit(self, dataframe: pd.DataFrame) -> 'MedianImputer':
        self.medians = dataframe.median(numeric_only=True).fillna(0)
        return self

    def transform(self, dataframe: pd.DataFrame) -> pd.DataFrame:
        return dataframe.fillna(self.medians)
```

A custom MedianImputer is used instead of sklearn's SimpleImputer to keep the preprocessing code beginner-friendly and easy to explain without complex dependencies.

CHAPTER 13

Machine Learning Basics

Core concepts needed to understand the AI model

13.1 What Is Machine Learning?

Machine learning (ML) is a method of programming where a computer learns patterns from data examples rather than following manually written rules. Instead of writing 'IF packet_count > 10000 AND duration < 1 THEN ATTACK', we give the computer thousands of examples of both BENIGN and ATTACK flows and let it discover the rules automatically.

13.2 Key Terminology

Term	Simple Explanation	In AI-NIDS
Dataset	A collection of examples	CICIDS2017 CSV file — 216,387 flow records
Feature	An input variable / column	Flow Duration, Destination Port, Bytes/s...
Label / Target	The correct answer for each example	0=BENIGN or 1=ATTACK
Training	Model learns patterns from training data	75% of dataset (162,290 rows)
Testing	Evaluate model on unseen data	25% of dataset (54,097 rows)
Model	The learned mathematical function	RandomForestClassifier object
Prediction	Model's output for new data	'BENIGN' or 'ATTACK'
Confidence	How certain the model is (0.0-1.0)	predict_proba() output
Classification	Task: assign a label to each input	Binary: BENIGN vs ATTACK
Overfitting	Model memorizes training data, fails on new data	avoided by test split and balanced weights

13.3 Training vs Prediction

```

TRAINING (done ONCE with ml/train.py):
CICIDS2017 Data → Preprocessing → Features + Labels
→ RandomForestClassifier.fit(X_train, y_train)
→ Saves: random_forest_model.joblib

```

```

PREDICTION (done for EVERY new live flow):
New Flow Features → PredictionService.predict()
→ model.predict([features]) → 0 or 1
→ model.predict_proba() → confidence

```

→ 'BENIGN' or 'ATTACK'

[NOTE] Training is a slow one-time process (minutes). Prediction is fast (milliseconds per flow).
AI-NIDS ships with a pre-trained model — you only need to retrain if you change the dataset.

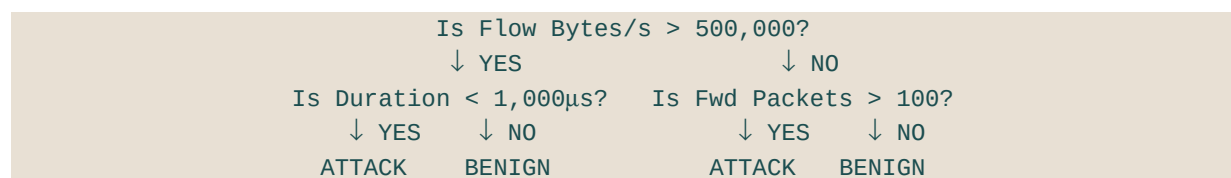
CHAPTER 14

Random Forest — The ML Algorithm

How AI-NIDS's brain makes decisions

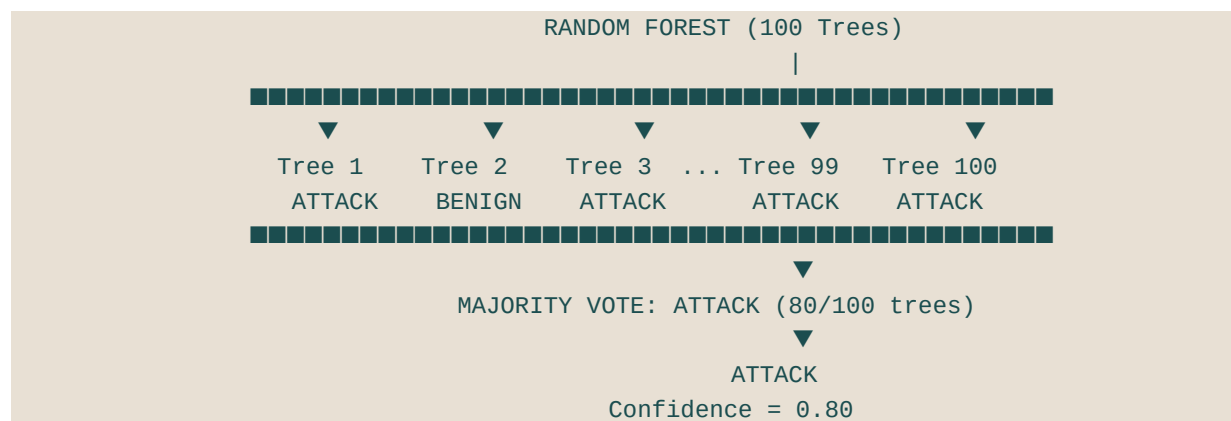
14.1 What Is a Decision Tree?

A **Decision Tree** is a model that makes predictions by asking a series of yes/no questions about the input features, following a tree-like structure until it reaches a leaf node (the prediction). Imagine a doctor's diagnosis flowchart.



14.2 What Is Random Forest?

A **Random Forest** is an ensemble of many decision trees. Each tree is trained on a random subset of the training data and uses a random subset of features at each split. The final prediction is made by **majority voting** across all 100 trees.



14.3 Why Random Forest for This Project?

- **Handles non-linear relationships** — Attack patterns are not simple linear thresholds.
- **No feature scaling required** — Random Forest works with raw values; no normalization needed.
- **Provides probability estimates** — `predict_proba()` gives confidence scores used for severity.
- **Robust to noise** — Ensemble voting reduces impact of individual tree errors.
- **Explainable** — Feature importance can be extracted to explain why a flow was classified as attack.

- **Well-understood algorithm** — Appropriate for an MCA academic project.

14.4 Actual Training Configuration

```
model = RandomForestClassifier(  
    n_estimators=100,          # 100 decision trees in the forest  
    random_state=42,          # Fixed seed for reproducibility  
    class_weight='balanced',  # Compensate for BENIGN/ATTACK imbalance  
    n_jobs=-1,                # Use all CPU cores for parallel training  
)  
model.fit(x_train_clean, y_train) # Train on 162,290 cleaned rows
```

Parameter explanations:

- **n_estimators=100**: Build 100 independent trees. More trees → more stable predictions. 100 is a good balance between accuracy and training speed.
- **random_state=42**: A fixed random seed so results are exactly reproducible every run.
- **class_weight='balanced'**: The dataset has more ATTACK rows (127,188) than BENIGN (89,199). Balanced weights make the model treat both classes equally during training.
- **n_jobs=-1**: Trains all 100 trees in parallel using all available CPU cores.

CHAPTER 15

Model Training

How the Random Forest model was built and saved

15.1 Training Command

```
python ml/train.py
```

Run from the project root with the virtual environment activated.

15.2 Training Steps — Actual Code Flow

```
train(data_path=DATA_DIR)
|
■■ load_csv_files(data_path) → pd.read_parquet() → raw DataFrame
■■ clean_dataset(raw_data) → strip, select, dedupe, numeric, NaN, labels
■■ split_features_and_label() → X (features), y (0/1 target)
■■ train_test_split(X, y, test_size=0.25, stratify=y)
■■ fit_imputer(x_train) → learn medians
■■ apply_imputer(x_train/x_test) → fill NaN values
■■ RandomForestClassifier.fit(x_train_clean, y_train) ← TRAINING
■■ model.predict(x_test_clean) → y_pred
■■ build_evaluation_text(y_test, y_pred) → accuracy, precision, recall, F1
■■ joblib.dump(model, MODEL_PATH) → random_forest_model.joblib
■■ joblib.dump(SELECTED_FEATURES, FEATURE_LIST_PATH) → feature_list.joblib
■■ joblib.dump({imputer, label_mapping}, PREPROCESSOR_PATH) → preprocessing_info.joblib
```

15.3 Saved Model Files

File	Contains	Used By
random_forest_model.joblib	Trained RandomForestClassifier with 100 decision trees	PredictionService.__init__() on application start
feature_list.joblib	List of 10 feature names in training order	arrange_features() — ensures correct column order
preprocessing_info.joblib	MedianImputer fitted on training data, label_mapping	apply_imputer to live flow features before prediction
evaluation_results.txt	Human-readable accuracy, precision, recall, F1, Classification report	ClassificationRoute → read_model_metrics() →

CHAPTER 16

Model Evaluation

How well the model performs — real results from evaluation_results.txt

16.1 Evaluation Metrics Explained

Accuracy

Percentage of all predictions that were correct.

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

Precision

Of all flows predicted as ATTACK, what fraction were actually attacks?

$$\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$$

High precision means few false alarms. Low precision means many legitimate flows are wrongly flagged as attacks.

Recall (Sensitivity)

Of all actual ATTACK flows, what fraction did the model catch?

$$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$$

High recall means the model catches most real attacks. Low recall means many real attacks are missed.

F1-Score

Harmonic mean of Precision and Recall — balances both.

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

16.2 Actual Results from evaluation_results.txt

[ACTUAL PROJECT] These are the ACTUAL results from ml/model/evaluation_results.txt generated by ml/train.py.

Metric	Score	Interpretation
Accuracy	0.9997 (99.97%)	Correct on 54,080 of 54,097 test flows
Precision	0.9998 (99.98%)	When it says ATTACK, it's right 99.98% of the time
Recall	0.9996 (99.96%)	Detects 99.96% of all actual attacks
F1-Score	0.9997 (99.97%)	Near-perfect balance of precision and recall

16.3 Confusion Matrix

The confusion matrix shows exactly how many predictions were correct or wrong:

	PREDICTED BENIGN	PREDICTED ATTACK
ACTUAL BENIGN	22,295 (True Negatives ✓)	5 (False Positives ✗)
ACTUAL ATTACK	12 (False Negatives ✗)	31,785 (True Positives ✓)

Result Type	Count	Meaning
True Positive (TP)	31,785	ATTACK correctly identified as ATTACK
True Negative (TN)	22,295	BENIGN correctly identified as BENIGN
False Positive (FP)	5	BENIGN wrongly flagged as ATTACK (false alarm)
False Negative (FN)	12	ATTACK missed — classified as BENIGN (missed detection)

16.4 What Do These Numbers Mean Practically?

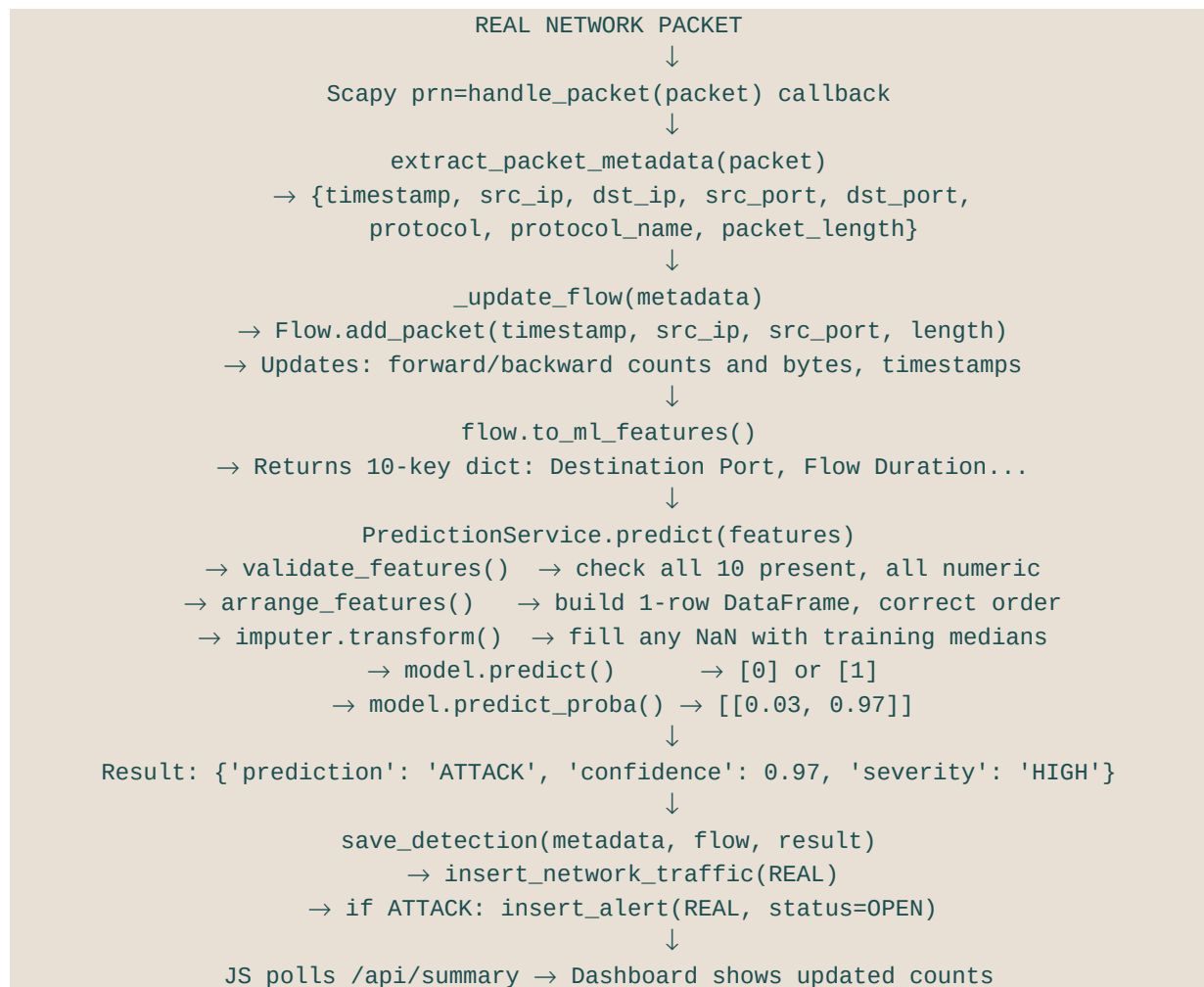
With only **5 false positives** and **12 false negatives** out of 54,097 test flows, the model is highly reliable for academic demonstration. In real production systems, the model would need retraining on diverse multi-year attack datasets, but for the CICIDS2017 Friday DDoS scenario, these results are excellent.

CHAPTER 17

From Live Packet to ML Prediction

The complete inference chain at runtime

17.1 The Complete Inference Pipeline



17.2 Feature Order Matters

The Random Forest model was trained with features in a specific order. If the live feature vector has columns in a different order, predictions will be wrong. This is why `arrange_features()` loads `feature_list.joblib` (the saved `SELECTED_FEATURES` list) and explicitly constructs the `DataFrame` in that exact order:

```
ordered_row = {feature: float(feature_values[feature]) for feature in self.feature_list}
dataframe = pd.DataFrame([ordered_row], columns=self.feature_list)
# Columns are now in EXACTLY the same order as during training
```

17.3 Known Limitation — Per-Packet vs Per-Flow Prediction

[IMPORTANT] IMPORTANT LIMITATION: AI-NIDS calls `to_ml_features()` and `predict()` on EVERY packet that updates a flow, not just at the end of the flow. This means early packets in a flow have very few statistics (e.g., Total Fwd Packets = 1, Flow Duration = 0). The model may produce less reliable predictions for very new flows. As more packets arrive and update the flow, predictions stabilize.

CHAPTER 18

Flask Backend

How the web server organizes routes, threads, and data

18.1 What Is Flask?

Flask is a lightweight Python web framework. It maps URLs (routes) to Python functions, renders HTML templates, and returns JSON responses. In AI-NIDS, Flask is both the web server that delivers the dashboard HTML and the API server that delivers data to JavaScript.

18.2 Complete Route Reference

[ACTUAL PROJECT] All routes are defined in app.py — these are the ACTUAL routes.

Route	Method	Purpose	Returns
/	GET	Main dashboard page	HTML (dashboard.html)
/live	GET	Live traffic full table	HTML (live_traffic.html)
/alerts	GET	Security alerts full table	HTML (alerts.html)
/model	GET	Model performance metrics	HTML (model.html)
/start-monitoring	POST	Start Scapy background thread	Redirect → /
/stop-monitoring	POST	Stop Scapy thread	Redirect → /
/load-demo	POST	Seed 10 demo traffic + 4 demo alert records	Redirect → /
/clear-demo	POST	DELETE FROM ... WHERE data_source='DEMO'	Redirect → /
/api/status	GET	Monitor thread active/stopped status	JSON
/api/summary	GET	KPI counts: packets, flows, attacks, alerts, has_demo_data	JSON
/api/live	GET	Latest 50 traffic records from SQLite	JSON array
/api/alerts	GET	Latest 50 alert records from SQLite	JSON array
/api/charts	GET	Aggregated chart datasets	JSON

18.3 Thread Safety with monitoring_lock

Flask runs in multiple threads. The packet capture also runs in a background thread. To prevent race conditions when reading/writing monitor_status and monitor_capture, all modifications use a threading.Lock:

```
monitor_lock = threading.Lock()

# Example: checking if already running
with monitor_lock:
    if monitor_thread and monitor_thread.is_alive():
        monitor_status['error'] = 'A capture session is already running.'
        return redirect(url_for('dashboard'))
```

18.4 read_model_metrics() — Parsing the Evaluation File

The /model route calls read_model_metrics() which reads evaluation_results.txt and uses regex to extract Accuracy, Precision, Recall, F1, and confusion matrix numbers. It includes a safe fallback dictionary returned if the file doesn't exist or parsing fails.

18.5 Flask Template Rendering

Flask uses Jinja2 for HTML templates. When dashboard() is called, it passes Python objects to the template:

```
return render_template(
    'dashboard.html',
    summary=get_dashboard_summary(DATABASE_PATH), # KPI numbers
    traffic=fetch_latest_traffic(DATABASE_PATH, 8), # 8 recent rows
    alerts=fetch_latest_alerts(DATABASE_PATH, 8), # 8 recent alerts
    status=monitor_status, # Active/stopped
    interfaces=get_interface_names(), # Dropdown list
    default_duration=CAPTURE_DURATION, # 10 seconds
)
```

CHAPTER 19

Database Design

SQLite schema and data management

19.1 Why SQLite?

- Zero configuration — no server installation required.
- Single file (nids.db) — easy to backup, delete, or inspect.
- Built into Python's standard library — no extra installation.
- Appropriate scale for an academic project with thousands of records.

19.2 Table: network_traffic

Column	Type	Description	Example Value
id	INTEGER PK AUTO	Unique row identifier	1
timestamp	TEXT NOT NULL	Flow capture time (ISO format)	2026-08-30 20:51:30
source_ip	TEXT	Origin host IP address	192.168.1.105
destination_ip	TEXT	Target host IP address	104.21.52.12
source_port	INTEGER	Originating port number	52410
destination_port	INTEGER	Target service port	443
protocol	TEXT	Protocol name (TCP/UDP/ICMP)	TCP
packet_count	INTEGER	Total packets in flow	148
byte_count	INTEGER	Total bytes in flow	124500
prediction	TEXT	ML result: BENIGN or ATTACK	BENIGN
confidence	REAL	Prediction probability (0.0-1.0)	0.9850
data_source	TEXT DEFAULT 'REAL'	Origin: 'REAL' or 'DEMO'	REAL

19.3 Table: alerts

Column	Type	Description	Example Value
id	INTEGER PK AUTO	Unique row identifier	1
timestamp	TEXT NOT NULL	Alert generation time	2026-08-30 20:51:30

source_ip	TEXT	Attack source IP	45.33.32.156
destination_ip	TEXT	Target IP	192.168.1.105
attack_type	TEXT	Always 'ATTACK'	ATTACK
confidence	REAL	Model confidence score	0.9940
severity	TEXT	HIGH (≥ 0.90) or MEDIUM (< 0.90)	HIGH
status	TEXT	Alert state: OPEN / REVIEWED / CLOSED	OPEN
data_source	TEXT DEFAULT 'REAL'	Origin: 'REAL' or 'DEMO'	REAL

19.4 Schema Auto-Migration

When `create_database()` is called at startup, it checks whether the `data_source` column already exists using `PRAGMA table_info`. If an older database is present without this column, it is added automatically without losing any existing data:

```
for table in ['network_traffic', 'alerts']:
    columns = [row['name'] for row in cursor.execute(f'PRAGMA table_info({table})')]
    if 'data_source' not in columns:
        cursor.execute(f'ALTER TABLE {table} ADD COLUMN data_source TEXT DEFAULT "REAL"')
```

19.5 Key SQL Queries Used

```
-- Dashboard KPI summary
SELECT COALESCE(SUM(packet_count),0) AS total_packets,
       COUNT(DISTINCT src||':'||sp||'>'||dst||':'||dp||'/'||proto) AS total_flows,
       SUM(CASE WHEN prediction='BENIGN' THEN 1 ELSE 0 END) AS normal_traffic,
       SUM(CASE WHEN prediction='ATTACK' THEN 1 ELSE 0 END) AS detected_attacks
FROM network_traffic;

-- Clear only demo data (never touches REAL records)
DELETE FROM network_traffic WHERE data_source = 'DEMO';
DELETE FROM alerts WHERE data_source = 'DEMO';
```

CHAPTER 20

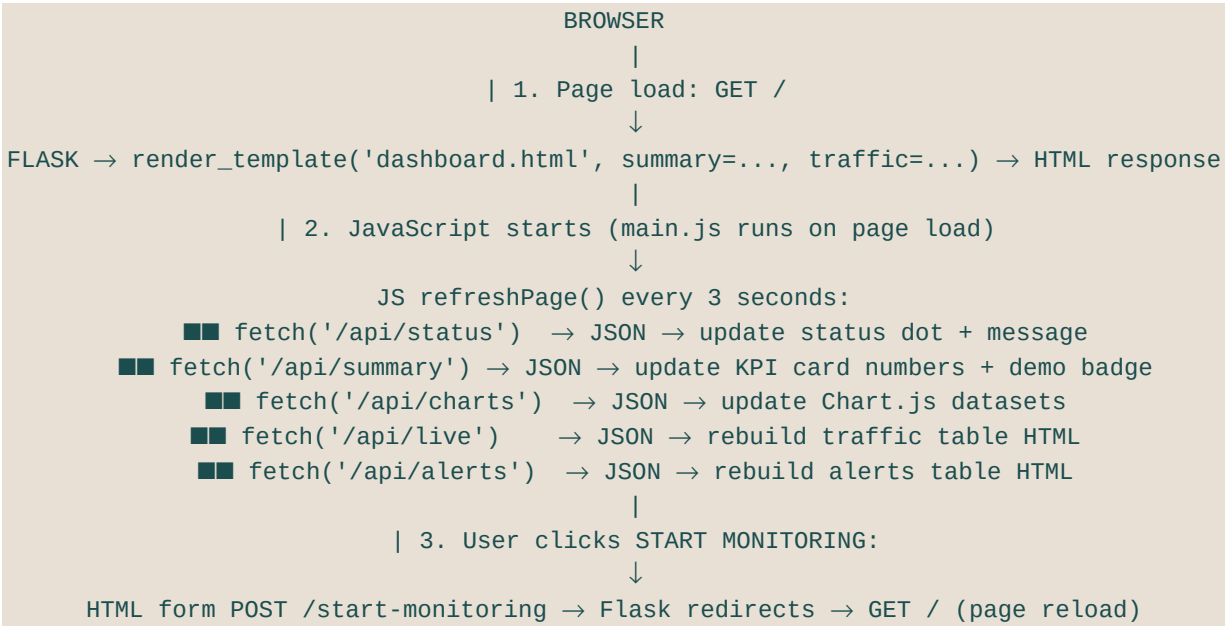
Frontend Architecture

How the dashboard is built and how it communicates with Flask

20.1 Technology Stack

Technology	Role	File
Jinja2 (Flask templates)	Server-side HTML generation	templates/*.html
Vanilla CSS	Retro-Futuristic styling	static/css/style.css
Bootstrap 5 (CDN)	Responsive grid and utilities	templates/base.html
Vanilla JavaScript	Polling, dynamic updates	static/js/main.js
Chart.js (CDN)	Doughnut, Line, Bar charts	Referenced in base.html

20.2 Frontend-Backend Communication Flow



20.3 Jinja2 Templates Structure

Template	Route	Content
base.html	All pages (extended)	Topbar nav, CSS/JS imports, live clock, status dot

dashboard.html	/	KPI cards, controls panel, charts, recent traffic/alert tables
live_traffic.html	/live	Full traffic table with 100 rows
alerts.html	/alerts	Full alerts table with severity badges
model.html	/model	Accuracy/Precision/Recall/F1 cards, confusion matrix, raw report

20.4 Design Palette

CSS Variable	Hex Color	Usage
Sand background	#F4EFE6	Page background
Parchment cards	#EFE7D8	Panel backgrounds
Dark Teal	#1B4D4F	Headers, nav, BENIGN badges
Terracotta Rust	#B84A2A	ATTACK badges, alerts, borders
Amber	#C28829	Demo mode badge, MEDIUM severity
Dark Brown	#2B2520	Primary text color

20.5 JavaScript Polling — Actual Timing

```
// main.js – bottom of file
updateLiveClock();           // Run clock immediately
setInterval(updateLiveClock, 1000); // Update clock every 1 second

refreshPage();               // Load all data immediately on page load
setInterval(refreshPage, 3000); // Refresh all data every 3 seconds
```

CHAPTER 21

Dashboard Components

Every element explained: what it shows, where the data comes from

21.1 Topbar Navigation

- **AI-NIDS logo/brand:** Displays project title.
- **Nav links:** Dashboard / Live Traffic / Alerts / Model Performance.
- **Status dot:** Green = MONITORING ACTIVE, Red = MONITORING STOPPED. Updated via /api/status.
- **Live clock badge:** Current date and time, updated every second by updateLiveClock().

21.2 DEMO MODE ACTIVE Badge

A golden badge '■ DEMO MODE ACTIVE' appears in the directive banner when demo data is present. It is controlled by the has_demo_data field in /api/summary JSON response. JavaScript shows/hides the badge by toggling the CSS class 'd-none'.

21.3 Monitoring Control Panel

Control	Action	Backend Effect
Interface dropdown	Select network adapter	Sent as POST parameter to /start-monitoring
Duration input (seconds)	Set capture time (5–3600s)	Sent as POST parameter
■ START MONITORING	POST /start-monitoring	Starts PacketCapture daemon thread
■ STOP	POST /stop-monitoring	Calls monitor_capture.stop() → sets stop_event
■ DEMO DATA	POST /load-demo	Calls seed_demo_data() — inserts 10 traffic + 4 alert rows
■ CLEAR DEMO	POST /clear-demo	Calls clear_demo_data() — DELETE WHERE data_source='DEMO'

21.4 KPI Cards

Card	Data Source	SQL Query Behind It
Total Packets	/api/summary → total_packets	SUM(packet_count) FROM network_traffic
Total Flows	/api/summary → total_flows	COUNT(DISTINCT 5-tuple)
Normal Traffic	/api/summary → normal_traffic	SUM(CASE WHEN prediction='BENIGN')

Detected Attacks	/api/summary → detected_attacks	SUM(CASE WHEN prediction='ATTACK')
Active Alerts	/api/summary → active_alerts	COUNT(*) FROM alerts WHERE status='OPEN'

21.5 Charts

Chart	Type	Data Source	What It Shows
Benign vs Attack	Doughnut	/api/charts → benign_attack	Proportion of BENIGN vs ATTACK flows. Teal=BENIGN
Traffic Over Time	Line	/api/charts → traffic_over_time	Number of flows per minute (last 10 minutes). Shows tr
Alert Severity	Bar	/api/charts → attack_count	Count of HIGH vs MEDIUM severity alerts. Rust=HIGH

CHAPTER 22

Alert System

When and how security alerts are generated and stored

22.1 What Triggers an Alert?

An alert is created **ONLY** when the Random Forest model predicts **ATTACK**. BENIGN predictions generate a traffic record but no alert. This logic is in `save_detection()` in `app.py`:

```
if result.get('prediction') == 'ATTACK':
    insert_alert(
        DATABASE_PATH,
        {
            'timestamp':    timestamp,
            'source_ip':    flow.source.ip,
            'destination_ip': flow.destination.ip,
            'attack_type':   'ATTACK',
            'confidence':    confidence,
            'severity':      result.get('severity', 'MEDIUM'),
            'status':        'OPEN',
        },
        data_source='REAL',
    )
```

22.2 Severity Assignment

Severity is determined by `calculate_severity()` in `ml/predict.py`:

```
def calculate_severity(prediction: str, confidence: float | None) -> str:
    if prediction == 'BENIGN':
        return 'No alert'
    if confidence is not None and confidence >= 0.90: # HIGH_CONFIDENCE_THRESHOLD
        return 'HIGH'
    return 'MEDIUM'
```

The threshold 0.90 is defined in `config.py` as `HIGH_CONFIDENCE_THRESHOLD`. It can be changed there without modifying any other code.

22.3 Alert Status Flow

```
ATTACK predicted → INSERT alert with status='OPEN'
                    |
                    (Future manual review)
                    ↓
                    status='REVIEWED'
```

↓
status='CLOSED'

Note: Status change UI is a FUTURE SCOPE feature.
All current alerts remain as 'OPEN'.

22.4 Alert Information Displayed

Field	Value Example	Meaning
Timestamp	2026-08-30 20:51:30	When the attack flow was detected
Source IP	45.33.32.156	IP address of the suspected attacker
Destination IP	192.168.1.105	IP address of the victim machine
Attack Type	ATTACK	Always 'ATTACK' in current version
Confidence	0.9940	Model certainty: 99.4%
Severity	HIGH	HIGH (>=90%) or MEDIUM (<90%)
Status	OPEN	Alert has not yet been reviewed

CHAPTER 23

Real Monitoring Mode

Step-by-step guide to live packet capture and classification

23.1 Prerequisites

- Application started via `run_ai_nids.bat` or `python run.py`.
- Npcap installed (`conf.use_pcap = True` verified after installation).
- ML model files present in `ml/model/` (`random_forest_model.joblib` etc.).
- Running on an authorized network that you own or have permission to monitor.

23.2 Step-by-Step Usage

1. Open Dashboard

Navigate to `http://127.0.0.1:5000`. The status dot is Red (STOPPED).

2. Select Interface

In the Interface dropdown, select your active adapter. Typically 'Wi-Fi' for wireless or 'Ethernet' for wired. The list comes from `get_interface_names()` which reads Scapy's `conf.ifaces`.

3. Set Duration

Enter how many seconds to capture (default: 10 from `config.py CAPTURE_DURATION`). For testing, 15–30 seconds gives enough time to generate multiple flows.

4. Click ■ START MONITORING

A POST request goes to `/start-monitoring`. Flask validates the inputs and starts a background daemon thread. The status dot turns Green.

5. Generate Network Activity

Open a browser tab and visit websites (HTTPS to port 443 → likely BENIGN). The Scapy sniffer captures these packets and processes them into flows.

6. Observe Live Updates

Every 3 seconds, the dashboard automatically refreshes. KPI cards update, new rows appear in the Recent Traffic table, and the Benign vs Attack chart updates its proportions.

7. Wait for Capture to Complete

After the duration expires, Scapy stops automatically. The status dot returns to Red. The final summary reflects all captured flows.

8. Click ■ STOP (Optional Early Stop)

Clicking STOP before the timer expires calls `monitor_capture.stop()` which sets `stop_event = True`. Scapy's `stop_filter` checks this and exits the sniff loop.

9. View Full Records

Click 'LIVE TRAFFIC' in the topbar to see all recorded flows. Click 'ALERTS' to see any detected attacks. All records have `data_source='REAL'`.

23.3 What Happens in the Background

While the user browses the dashboard, the packet capture thread independently processes packets. Python's `threading.Lock` ensures that when `save_detection()` writes to SQLite and when the Flask API reads from SQLite, they don't collide. SQLite itself is thread-safe with `check_same_thread=False` enabled in `get_connection()`.

CHAPTER 24

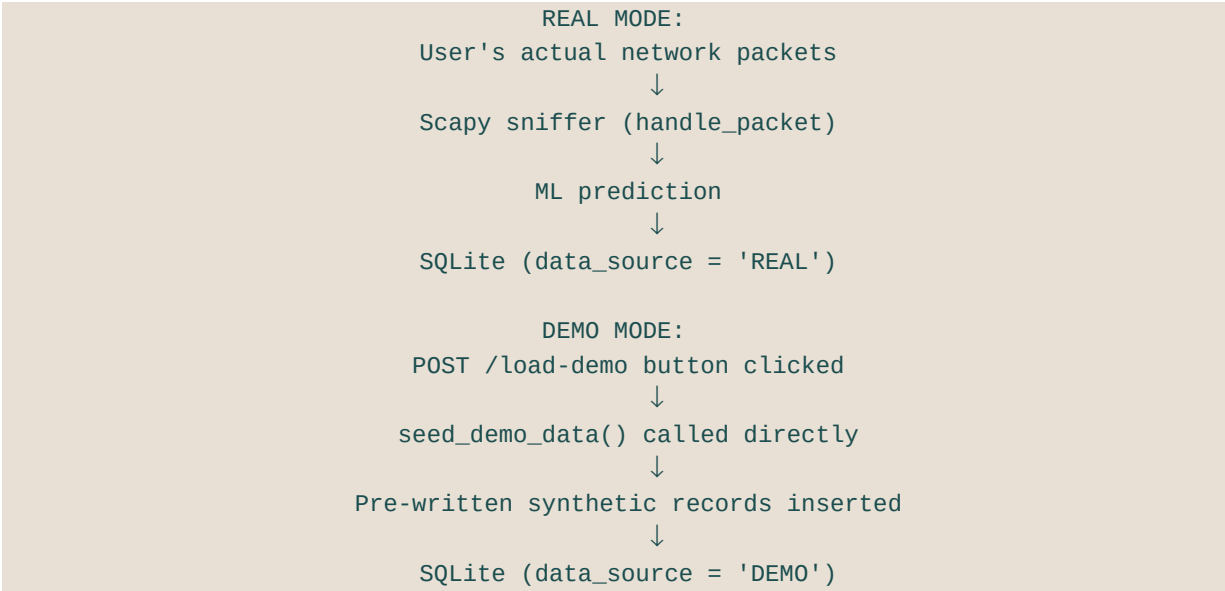
Demo Mode

Presenting the project without live network traffic

24.1 Why Demo Mode Exists

During a college presentation or project viva, generating real attack traffic is not practical or ethical. Demo Mode seeds the database with 10 realistic synthetic traffic records and 4 alert records that look exactly like real captured data. The dashboard then displays populated charts, tables, and KPI counts.

24.2 Demo Data vs Real Data — Side-by-Side



[IMPORTANT] Demo data records are NOT generated by the ML model. They are hardcoded realistic-looking values written directly into database/db.py. The confidence scores and predictions are manually specified, not computed.

24.3 The 10 Demo Traffic Records (Actual Data)

Source IP	Destination	Port	Proto	Packets	Prediction	Confidence
192.168.1.105	104.21.52.12	443	TCP	148	BENIGN	0.9850
192.168.1.105	8.8.8.8	53	UDP	12	BENIGN	0.9920
192.168.1.105	142.250.190.46	443	TCP	210	BENIGN	0.9780
45.33.32.156	192.168.1.105	80	TCP	1250	ATTACK	0.9940

185.220.101.5	192.168.1.105	22	TCP	45	ATTACK	0.8850
192.168.1.105	13.107.42.14	443	TCP	84	BENIGN	0.9910
192.168.1.120	192.168.1.1	53	UDP	6	BENIGN	0.9960
198.51.100.44	192.168.1.105	8080	TCP	2150	ATTACK	0.9980
192.168.1.105	172.217.16.206	443	TCP	92	BENIGN	0.9890
103.21.244.0	192.168.1.105	445	TCP	32	ATTACK	0.8720

24.4 Clearing Demo Data

Clicking '■ CLEAR DEMO' sends POST to /clear-demo, which calls `clear_demo_data()`. This executes: `DELETE FROM network_traffic WHERE data_source = 'DEMO'` and `DELETE FROM alerts WHERE data_source = 'DEMO'`. Records with `data_source = 'REAL'` are NEVER affected.

24.5 DEMO MODE ACTIVE Badge

`has_demo_data()` queries `COUNT(*) WHERE data_source='DEMO' > 0`. This boolean is included in /api/summary JSON. JavaScript's `refreshSummary()` calls `demoBadge.classList.remove('d-none')` when `has_demo_data` is true, making the golden badge visible.

CHAPTER 25

One-Click Launcher

run_ai_nids.bat and run.py explained

25.1 run_ai_nids.bat — Windows Batch Launcher

```
@echo off
TITLE AI-NIDS Launcher
COLOR 0A                      :: Green text on black background
CD /D "%~dp0"                  :: Change to the directory of this .bat file

IF EXIST ".venv\Scripts\python.exe" (
    SET "PYTHON_EXE=.venv\Scripts\python.exe"  :: Use virtual environment
) ELSE (
    SET "PYTHON_EXE=python"                    :: Fall back to system Python
)

"%PYTHON_EXE%" run.py           :: Launch the Python runner script

IF ERRORLEVEL 1 (
    echo [ERROR] AI-NIDS failed to start.
    pause                        :: Keep window open so error is visible
)
```

25.2 run.py — Python Launcher

```
def main():
    print_banner()                # Print ASCII banner with paths
    create_database(DATABASE_PATH) # Ensure nids.db exists with correct schema

    url = f'http://{HOST}:{PORT}' # http://127.0.0.1:5000

    # Open browser BEFORE Flask's blocking app.run()
    browser_thread = threading.Thread(
        target=open_browser,
        args=(url, 1.5),          # Wait 1.5 seconds then open
        daemon=True
    )
    browser_thread.start()

    from app import app
    app.run(host=HOST, port=PORT, debug=False) # Blocking call
```

25.3 Startup Sequence

```
graph TD; A[run_ai_nids.bat] --> B[.venv/Scripts/python.exe run.py]; B --> C[print_banner() - ASCII welcome message]; C --> D[create_database(DATABASE_PATH) - nids.db initialized / migrated]; D --> E[browser_thread starts (waits 1.5s then opens http://127.0.0.1:5000)]; E --> F[app.run(host='127.0.0.1', port=5000, debug=False) - Flask starts serving]; F --> G[Browser opens dashboard automatically]; G --> H[Application ready];
```

run_ai_nids.bat

↓

.venv/Scripts/python.exe run.py

↓

print_banner() – ASCII welcome message

↓

create_database(DATABASE_PATH) – nids.db initialized / migrated

↓

browser_thread starts (waits 1.5s then opens http://127.0.0.1:5000)

↓

app.run(host='127.0.0.1', port=5000, debug=False) – Flask starts serving

↓

Browser opens dashboard automatically

↓

Application ready

CHAPTER 26

Project File Structure

Every file explained

26.1 Directory Tree

```

AI-NIDS/
├── app.py           ← Flask application (routes, thread management)
├── config.py        ← All configurable constants (paths, host, port)
├── run.py           ← Python launcher (banner, DB init, browser open)
├── run_ai_nids.bat  ← Windows one-click launcher (double-click to run)
├── requirements.txt ← Python package dependencies
├── README.md        ← Project overview and setup guide
├── .gitignore       ← Git ignore rules (.venv, nids.db, .log files)
├──
├── database/
│   ├── __init__.py
│   ├── db.py        ← All SQLite functions (create, insert, fetch, demo)
│   └── models.py     ← (Unused schema reference placeholder)
│   └──
├── ml/
│   ├── __init__.py
│   ├── preprocess.py ← SELECTED_FEATURES, MedianImputer, clean_dataset()
│   └── train.py       ← Training script: load → clean → train → evaluate → save
│   └──
├── predict.py       ← PredictionService: loads model, predicts BENIGN/ATTACK
├── model/
│   ├── random_forest_model.joblib ← Trained RandomForestClassifier
│   ├── feature_list.joblib        ← Ordered list of 10 feature names
│   ├── preprocessing_info.joblib  ← MedianImputer + label_mapping
│   └── evaluation_results.txt      ← Accuracy, F1, confusion matrix
│   └──
├── network/
│   ├── __init__.py
│   ├── features.py ← extract_packet_metadata() + flow key functions
│   ├── flow.py     ← Flow class: bidirectional stats + to_ml_features()
│   └── packet_capture.py ← PacketCapture class: Scapy sniff + prediction wire
│   └──
├── data/
│   ├── Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv.parquet
│   └── README.md
│   └──
├── database/
│   └── nids.db ← SQLite database (gitignored; created on first run)
│   └──
└── static/

```

```
■ ■■■ css/style.css      ← Custom retro-futuristic CSS theme
■ ■■■ js/main.js        ← All JavaScript: polling, charts, clock, tables
■
■ ■■■ templates/
■ ■■■ base.html         ← Base template with topbar, nav, clock, imports
■ ■■■ dashboard.html    ← Main dashboard: KPIs, controls, charts, tables
■ ■■■ live_traffic.html  ← Full 100-row traffic table
■ ■■■ alerts.html       ← Full 100-row alerts table
■ ■■■ model.html        ← Model performance: accuracy cards, confusion matrix
■
■ ■■■ tests/
■ ■■■ test_pipeline.py   ← Flow feature extraction unit test
■ ■■■ test_prediction_service.py ← Prediction service unit test
■
■ ■■■ docs/
■ ■■■ PROJECT_DOCUMENTATION.md
■ ■■■ SYSTEM_ARCHITECTURE.md
■ ■■■ DATABASE_DESIGN.md
■ ■■■ ML_PIPELINE.md
■ ■■■ USER_GUIDE.md
■ ■■■ DEVELOPMENT_HISTORY.md
```

CHAPTER 27

Code Walkthrough

Most important functions explained line by line

27.1 network/features.py — extract_packet_metadata()

```
def extract_packet_metadata(packet) -> dict | None:
    if IP not in packet:          # Skip non-IP frames (ARP, etc.)
        return None

    ip_layer = packet[IP]
    protocol_number = int(ip_layer.proto) # 1=ICMP, 6=TCP, 17=UDP
    source_port = 0
    destination_port = 0

    if TCP in packet:
        source_port = int(packet[TCP].sport)
        destination_port = int(packet[TCP].dport)
    elif UDP in packet:
        source_port = int(packet[UDP].sport)
        destination_port = int(packet[UDP].dport)
    elif ICMP in packet:
        source_port = 0          # ICMP has no ports
        destination_port = 0

    return {
        'timestamp':      float(packet.time),      # Unix epoch timestamp
        'source_ip':      ip_layer.src,
        'destination_ip': ip_layer.dst,
        'source_port':    source_port,
        'destination_port': destination_port,
        'protocol':       protocol_number,
        'protocol_name':  PROTOCOL_NAMES.get(protocol_number, str(protocol_number))
    },
    {
        'packet_length':  len(packet),             # Total frame size in bytes
    }
}
```

27.2 network/flow.py — add_packet()

```
def add_packet(self, timestamp: float, source_ip: str,
               source_port: int, packet_length: int) -> None:
    if self.start_time is None:
        self.start_time = timestamp          # Record first packet time
```

```

self.last_time = timestamp          # Always update last packet time

# Determine direction: is this packet from the original source?
is_forward = (source_ip == self.source.ip and
              source_port == self.source.port)
if is_forward:
    self.forward_packet_count += 1
    self.forward_byte_count += packet_length
else:
    self.backward_packet_count += 1
    self.backward_byte_count += packet_length

```

27.3 ml/predict.py — PredictionService.predict()

```

def predict(self, feature_values: dict) -> dict:
    # Step 1: Build correctly ordered 1-row DataFrame
    dataframe = self.arrange_features(feature_values)
    # - validate all 10 features present and numeric
    # - order columns to match training order
    # - apply MedianImputer for any NaN

    # Step 2: Model predicts class label
    prediction_number = int(self.model.predict(dataframe)[0]) # 0 or 1
    prediction = 'ATTACK' if prediction_number == 1 else 'BENIGN'

    # Step 3: Model returns probability for each class
    confidence = self._confidence(dataframe, prediction_number)
    # e.g., [0.03, 0.97] → confidence for ATTACK = 0.97

    # Step 4: Determine severity based on confidence threshold
    return {
        'prediction': prediction,
        'confidence': confidence,
        'severity': calculate_severity(prediction, confidence)
    }

```

27.4 database/db.py — insert_network_traffic()

```

def insert_network_traffic(database_path, record, data_source='REAL'):
    with get_connection(database_path) as connection:
        connection.execute(
            'INSERT INTO network_traffic (timestamp, source_ip, destination_ip,
            source_port, destination_port, protocol, packet_count, byte_count,
            prediction, confidence, data_source) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)',
            (
                record['timestamp'],
                record['source_ip'],
                record['destination_ip'],
                record['source_port'],

```

```
        record['destination_port'],
        record['protocol'],
        record['packet_count'],
        record['byte_count'],
        record['prediction'],
        record['confidence'],
        data_source,          # 'REAL' or 'DEMO'
    )
)
```

CHAPTER 28

Complete End-to-End Example

Tracing one real packet from capture to dashboard

28.1 Scenario: Your laptop browses <https://www.google.com>

Let's trace exactly what happens when your browser makes an HTTPS request to Google while AI-NIDS monitoring is active.

1 — Browser sends HTTPS request

Your browser sends a TCP packet:

```
src: 192.168.1.105:52432 → dst: 142.250.190.46:443 (Google)
Protocol: TCP (6), Length: 512 bytes
```

2 — Scapy captures the packet

Scapy's sniff() callback fires: handle_packet(packet) is called.

3 — Metadata extracted

extract_packet_metadata(packet) returns:

```
timestamp=1756567212.342, src_ip='192.168.1.105', dst_ip='142.250.190.46'
src_port=52432, dst_port=443, protocol=6, protocol_name='TCP', length=512
```

4 — Flow identified

_update_flow() checks self.flows. If this is the first packet to :443, a new Flow is created with flow_key=(192.168.1.105, 142.250.190.46, 52432, 443, 6). add_packet() sets start_time=1756567212.342, forward_packet_count=1, forward_byte_count=512.

5 — Google's reply arrives

Google sends back: src=142.250.190.46:443 → dst=192.168.1.105:52432, length=1460 bytes. reverse_key matches the existing flow. add_packet() increments backward_packet_count=1, backward_byte_count=1460.

6 — Features extracted

```
to_ml_features() computes (assume duration=0.01s):
  Destination Port=443, Protocol=6, Flow Duration=10000µs
  Total Fwd Packets=1, Total Bwd Packets=1
  Total Length Fwd=512, Total Length Bwd=1460
  Fwd Mean=512.0, Bwd Mean=1460.0
  Flow Bytes/s=197200, Flow Packets/s=200
```

7 — Random Forest prediction

```
PredictionService.predict(features):
  → 1-row DataFrame with 10 columns in correct order
  → model.predict() → [0] (BENIGN)
  → model.predict_proba() → [[0.989, 0.011]]
  → confidence=0.989, prediction='BENIGN', severity='No alert'
```

8 — Database insert

```
save_detection() formats timestamp='2026-08-30 20:54:15'.
insert_network_traffic() INSERT INTO network_traffic ... prediction='BENIGN', confidence=0.989, data_source='REAL'.
Since prediction ≠ 'ATTACK', NO alert is inserted.
```

9 — Dashboard updates

```
JavaScript's 3-second interval fires:
  /api/summary returns total_packets += 1972, normal_traffic += 1
  /api/live returns the new traffic row
  Charts update with new BENIGN count
  The BENIGN slice of the doughnut grows slightly
```

CHAPTER 29

Error Handling

What happens when things go wrong

Problem	Cause	What AI-NIDS Does
Port 5000 already in use	Another Flask instance or Flask running on port 5000	Flask raises OSError. Flask.py catches it and prints 'Server error'. Kill old process with: kill -9 <code>PID</code>
Npcap not installed	WinPcap/Npcap driver missing	Scapy falls back to Layer-3 socket sniffing. If that also fails, error_message is set and capture ends.
Permission denied on interface	Not running as Administrator	Scapy raises OSError. error_message set. Run as Administrator or install Npcap properly.
ML model files missing	ml/train.py was never run	load_predictor() raises FileNotFoundError. prediction remains 'PENDING'. Fix: python train.py
Invalid interface selected	Interface name not in Scapy list	Flask validates interface in get_interface_names() before starting. Shows error message.
Invalid duration (<5 or >3600)	Out-of-range duration form	Flask check: if duration < 5 or > 3600, redirects with error message.
Monitoring started twice	User clicks START while already monitoring	Already monitoring check: if monitor_thread.is_alive(), rejects with 'already running' message.
Non-numeric ML features	Zero-duration flow or corrupted packet	validate_features() raises ValueError. Caught in handle_packet() exception block. Prediction remains 'PENDING'.
Database write fails	Disk full, permissions, or SQLite locked	Save_packet() wraps everything in try/except. Exception logged, monitor_status['error'] set. Capture ends.
No packets arriving	Interface wrong, no traffic, sniffing permissions	Sniff() times out. Capture ends with 0 packets. No error raised.

CHAPTER 30

Security & Privacy

How AI-NIDS protects privacy and the ethical rules for its use

[IMPORTANT] MANDATORY: AI-NIDS must ONLY be used on networks and devices you own or have explicit written authorization to monitor. Unauthorized network monitoring is illegal in most countries.

30.1 Privacy Protections Built Into the Code

Protection	Implementation	Effect
No payload storage	store=False in sniff(). features.py only extracts header fields	Raw packets containing passwords, web page content never written to disk.
Metadata only	extract_packet_metadata() discards packet object, exposing only metadata	Only IP addresses, ports, and packet size fields are stored.
No packet injection	AI-NIDS uses Scapy in read-only sniff() mode and does not inject() packets.	Cannot be used to launch attacks.
Local only	Flask bound to 127.0.0.1 (localhost). Data accessible from API only.	Not accessible from the internet.
Demo data is synthetic	Demo IP addresses (45.33.32.156, etc) are made up for presentation.	Do not make sense for presentation.

30.2 Disclaimer

This software is an educational tool developed for an MCA academic project. It is designed to demonstrate machine learning concepts applied to network security. It should only be run on personal equipment on networks you are authorized to use. The authors do not condone any unauthorized use of this software.

CHAPTER 31

Known Limitations

Honest assessment of what AI-NIDS cannot do

Limitation	Explanation
Binary classification only	Model predicts only BENIGN or ATTACK. It cannot tell you the specific attack type (port scan vs DDoS v
Trained on one dataset file	The model is trained only on Friday afternoon DDoS data from CICIDS2017. It may have reduced accur
Per-packet prediction limitation	Prediction is made after every packet update to the flow, not just at flow completion. Early-flow predictio
Encrypted traffic	HTTPS payload is encrypted. AI-NIDS can only see that an HTTPS connection exists and its volume/tim
Single interface monitoring	Monitors one interface at a time. Production NIDS systems monitor all network segments.
No alert management UI	Alerts cannot be marked as Reviewed or Closed through the UI. All alerts remain OPEN.
No authentication	Anyone with access to http://127.0.0.1:5000 can view the dashboard. No login required.
Passive monitoring only	Cannot block detected attacks. Cannot reset TCP connections. Detection only, no prevention.
Alert status 'OPEN' permanently	The 'REVIEWED' and 'CLOSED' status values are defined in the schema but no UI to change them.

CHAPTER 32

Future Improvements

Realistic enhancements that could be added next

[TIP] All items in this chapter are FUTURE SCOPE — not implemented in the current version.

Future Feature	Description	Complexity
Multiclass attack classification	Train on full CICIDS2017 with all 14 attack categories (PortScan, Denial of Service, etc.) on labeled data and export as CSV	Medium — requires labeled data and export functionality
WebSocket live updates	Replace polling (every 3 seconds) with WebSocket push for instant updates	Medium — Flask-SocketIO library
User authentication	Login page with username/password before accessing the dashboard	Low — Flask-Login or Flask-HTTPAuth
Alert management UI	Buttons to mark alerts as Reviewed/Closed. Status history tracking	Low — simple form + SQL UPDATE
PDF / CSV report export	Download all detected alerts as PDF or Excel report	Low — ReportLab or pandas to_csv()
Multi-interface monitoring	Monitor multiple network adapters simultaneously with separate dashboards	Medium — thread pool per interface
Automatic model retraining	Scheduled job to retrain model on new data collected from real traffic	High — requires labeled real traffic data
Anomaly detection	Unsupervised model (Isolation Forest / Autoencoder) to detect unknown attacks	High — different model architecture needed

CHAPTER 33

Complete Installation Guide

Setting up AI-NIDS from scratch

33.1 Prerequisites

- Python 3.10 or higher (<https://python.org/downloads>)
- Git (<https://git-scm.com/downloads>)
- Npcap (<https://npcap.com>) — install with 'WinPcap API-compatible mode' checked
- Windows 10/11 (or Linux/macOS with root access)

33.2 Installation Commands

```
# 1. Clone the GitHub repository
git clone https://github.com/vrajkiran/AI-NIDS.git
cd AI-NIDS

# 2. Create virtual environment
python -m venv .venv

# 3. Activate virtual environment (Windows)
.venv\Scripts\activate

# 4. Install all dependencies
pip install -r requirements.txt

# requirements.txt contains:
# Flask, scapy, pandas, numpy, scikit-learn, joblib, pyarrow

# 5. (Pre-trained model already in ml/model/ – skip training if model files exist)
# To retrain: python ml/train.py

# 6. Launch the application
python run.py
# OR double-click: run_ai_nids.bat
```

33.3 Verify Installation

```
# Run unit tests to confirm everything works
.venv\Scripts\python -m unittest discover tests

# Expected output:
# ....
```

```
# Ran 4 tests in 0.021s  
# OK
```

CHAPTER 34

How To Use the Application

Beginner-friendly step-by-step user guide

34.1 Launching the Application

- Double-click **run_ai_nids.bat** in the project folder.
- A terminal window opens showing the AI-NIDS banner.
- Your default browser opens automatically to `http://127.0.0.1:5000`.
- The dashboard loads with any existing records (or demo data).

34.2 Dashboard Controls Summary

Button / Control	What It Does
■ START MONITORING	Begins live Scapy packet capture with ML classification
■ STOP	Stops active monitoring (preserves existing records)
■ DEMO DATA	Loads 10 sample traffic + 4 alert records for presentation
■ CLEAR DEMO	Deletes ONLY demo records, REAL records are untouched
Interface dropdown	Select your active network adapter (Wi-Fi, Ethernet, etc.)
Duration input	Seconds to capture (5–3600). Default: 10 seconds
LIVE TRAFFIC nav	View all 100 most recent traffic flow records
ALERTS nav	View all 100 most recent security alerts
MODEL PERFORMANCE	View model accuracy, confusion matrix, classification report

34.3 For Project Demonstration

- Click ■ DEMO DATA to load realistic sample records instantly.
- The '■ DEMO MODE ACTIVE' badge appears in the header.
- Show KPI cards (Total Packets: 4029, Total Flows: 10, Detected Attacks: 4).
- Show the Doughnut chart (6 BENIGN : 4 ATTACK).
- Navigate to ALERTS page — show 4 alerts with HIGH/MEDIUM severity.
- Navigate to MODEL PERFORMANCE — show Accuracy 99.97%.
- Click ■ CLEAR DEMO to clean up after demonstration.

CHAPTER 35

Troubleshooting

Common problems and their solutions

Problem	Likely Cause	Solution
Browser doesn't open or shows Flashable toolbar	Flashable toolbar port 5000 is busy	Run <code>netstat -ano findstr :5000</code> . If found, <code>taskkill /PID <pid> /F</code> . Restart.
'WARNING: No libpcap provided for platform'	libpcap not installed or not configured	Reinstall Npcap and compatibility with 'WinPcap API-compatible mode' checked
No packets captured after clicking START	Wrong NI-PC interface selected, no network connectivity, different interface selected	Generate traffic (open browser). Run as Administrator
Dashboard shows 0 Flows / empty Tables	No PCAPs captures done and demo records cleared	Click Load Demo Data to reload demo records, or start real monitoring.
Model page shows N/A for all metrics	<code>evaluation_results.txt</code> missing	Run <code>modeltrainer.py</code> (requires CICIDS2017 data in <code>data/</code> folder)
'Missing required ML features'	Flow features don't match the <code>FEATURES</code> in <code>preprocess.py</code>	Ensure <code>FEATURES</code> in <code>preprocess.py</code> matches <code>to_ml_features()</code> in <code>modeltrainer.py</code>
'A capture session is already running'	Clicked START while monitoring is still active	Wait till active session to complete or click STOP first.
Git push fails with authentication error	GitHub token expired or gh CLI not authenticated	Run <code>gh auth login</code> and re-authenticate.
Tests fail: <code>ModuleNotFoundError</code>	Virtual environment not activated	Run: <code>.venv\Scripts\activate</code> first, then run tests.
Charts are empty / not rendering	Chart.js CDN not loading (offline) or JavaScript console errors	Check JavaScript console (F12) for errors. Ensure internet connection for CDN.

CHAPTER 36

Project Development Journey

How the project was built from idea to final product

Phase 1 — Planning & Architecture

Defined the 5-component architecture: Scapy → Flow → ML → SQLite → Flask. Selected CICIDS2017 dataset for training. Selected Random Forest for its interpretability and strong performance on tabular flow data.

Phase 2 — Configuration Setup

Created config.py with all file paths (BASE_DIR, MODEL_PATH, DATABASE_PATH, etc.), runtime constants (CAPTURE_DURATION=10, HIGH_CONFIDENCE_THRESHOLD=0.90), and server config (HOST='127.0.0.1', PORT=5000).

Phase 3 — Dataset & Preprocessing (ml/preprocess.py)

Defined SELECTED_FEATURES (10 live-calculable features), implemented load_csv_files() with Parquet support, clean_dataset() pipeline, convert_labels() (BENIGN→0, other→1), and custom MedianImputer class.

Phase 4 — Model Training (ml/train.py)

Implemented train() function: 75/25 stratified split, RandomForestClassifier(n_estimators=100, class_weight='balanced'), evaluation with sklearn metrics, saving 3 Joblib files + evaluation_results.txt.

Phase 5 — Prediction Service (ml/predict.py)

Implemented PredictionService: loads model on startup, validate_features() checks for missing/NaN/Inf values, arrange_features() preserves column order, predict() returns {prediction, confidence, severity}.

Phase 6 — Packet Capture Engine (network/features.py + flow.py + packet_capture.py)

Built extract_packet_metadata() for safe header extraction, Flow class with bidirectional tracking and to_ml_features(), PacketCapture with Scapy sniff(), Layer-2/3 fallback, and stop_event threading.

Phase 7 — Database Layer (database/db.py)

Designed SQLite schema for network_traffic and alerts tables, implemented get_connection(), create_database() with auto-migration, insert/fetch functions, dashboard summary query, and chart data queries.

Phase 8 — Flask Backend (app.py)

Connected all components: Flask routes, threading with monitor_lock, save_detection() callback, read_model_metrics() with regex parsing, all API endpoints (/api/status, /api/summary, /api/live, /api/alerts, /api/charts).

Phase 9 — Frontend & Dashboard (templates/ + static/)

Built Jinja2 templates with base.html inheritance. Created CSS with Retro-Futuristic parchment/teal/rust palette. Implemented JavaScript polling, Chart.js initialization and update, live clock, and table rendering.

Phase 10 — Demo Mode & Data Separation

Added data_source column with schema auto-migration, seed_demo_data() with 10 realistic synthetic records, clear_demo_data() selective deletion, has_demo_data() check, /load-demo and /clear-demo routes, ■ DEMO MODE ACTIVE badge with JavaScript toggle.

Phase 11 — Launcher & GitHub

Created run_ai_nids.bat (Windows batch, auto .venv detection) and run.py (print_banner, create_database, browser_thread, app.run). Git initialized, .gitignore created (excluding .venv, nids.db, logs, installer). Pushed to <https://github.com/vrajikiran/AI-NIDS>.

Phase 12 — Testing & Documentation

Wrapped unit tests in unittest.TestCase. Ran 4/4 tests passing. Created 6 documentation files in docs/. Generated this PDF guide.

CHAPTER 37

Quick Revision

1-minute and 5-minute explanations + complete glossary

37.1 One-Minute Explanation

[ACTUAL PROJECT] AI-NIDS watches your computer's network, groups packets into conversations (flows), calculates 10 statistics per flow, feeds them to a Random Forest ML model trained on CICIDS2017 DDoS data, and classifies each flow as BENIGN or ATTACK with 99.97% accuracy. Results appear live on a Flask web dashboard at <http://127.0.0.1:5000>.

37.2 Five-Minute Explanation

The Problem: Networks are constantly being attacked. Traditional tools use fixed rules that miss new attacks. Humans can't inspect millions of packets.

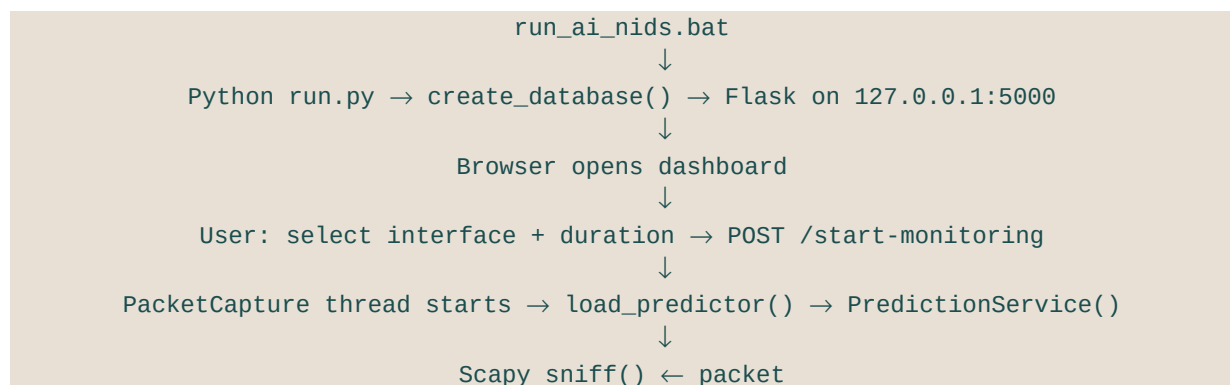
The Solution: AI-NIDS uses machine learning. We trained a Random Forest model on 225,745 labeled network flows from the CICIDS2017 dataset (BENIGN and DDoS attacks). The model learned patterns: attacks have very high packet rates, unusual byte volumes, and specific port/protocol signatures.

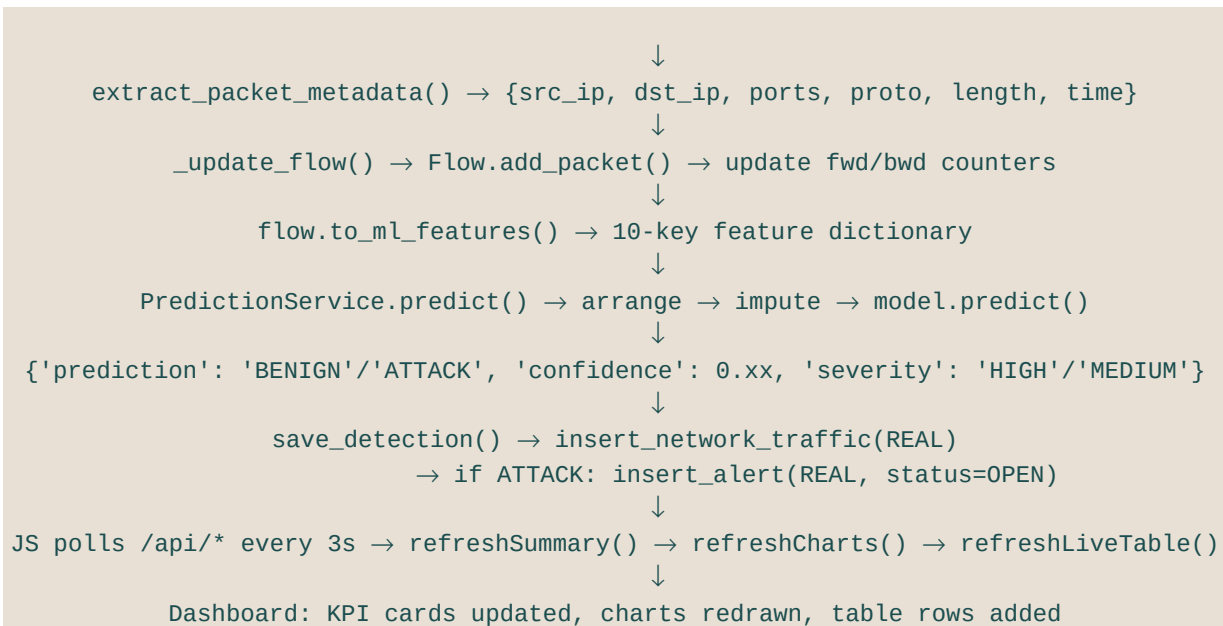
Live Operation: Scapy captures packet headers (never payload) from authorized network adapters. Packets are grouped into bidirectional 5-tuple flows. 10 statistical features are computed per flow. The trained model predicts BENIGN or ATTACK with a confidence score in milliseconds.

Storage: Every flow and every alert is stored in SQLite with a data_source tag ('REAL' for live captures, 'DEMO' for presentation data). The database is never exposed to the internet.

Dashboard: A Flask web server exposes REST API endpoints. JavaScript polls these every 3 seconds and updates Chart.js visualizations and HTML tables without page reloads. The Model Performance page shows the actual accuracy (99.97%) from the evaluation file.

37.3 Complete Technical Flow Diagram





37.4 Glossary of Key Terms

Term	Definition
Packet	A small chunk of data sent over a network, containing header + payload
Flow	A group of packets belonging to the same bidirectional network conversation
5-tuple	The 5 fields that uniquely identify a flow: srcIP, dstIP, srcPort, dstPort, protocol
IP Address	Numeric label (e.g., 192.168.1.105) identifying a device on a network
Port	A numbered channel on a device for a specific service (443=HTTPS, 22=SSH)
Protocol	Communication rules: TCP=6, UDP=17, ICMP=1
Scapy	Python library for passive packet capture (sniffing) from network adapters
Npcap	Windows kernel driver enabling Scapy to capture raw network packets
Feature	A numeric input variable for the ML model (e.g., Flow Bytes/s)
CICIDS2017	Canadian ICS benchmark dataset with labeled BENIGN and attack network flows
Random Forest	Ensemble ML algorithm of 100 decision trees. Final prediction by majority vote
Joblib	Python library for saving/loading large ML model objects efficiently
MedianImputer	Custom class that fills missing values with the median of training data
predict_proba()	sklearn method returning probability for each class (confidence score)
Confidence	Probability (0.0–1.0) that the prediction is correct

SQLite	Serverless file-based relational database (nids.db)
data_source	SQLite column: 'REAL' = live captured flow, 'DEMO' = synthetic demo record
Flask	Python web framework mapping URLs to Python functions
Jinja2	HTML templating engine for Flask (used in templates/*.html)
REST API	HTTP interface that returns JSON data (/api/summary, /api/live, etc.)
Chart.js	JavaScript library for interactive Doughnut, Line, and Bar charts
Polling	JavaScript repeatedly calling /api/* every 3 seconds for fresh data
Demo Mode	Presentation mode using synthetic data_source='DEMO' records
BENIGN	ML classification: normal, expected network traffic
ATTACK	ML classification: traffic matching learned DDoS/attack patterns
Severity HIGH	ATTACK prediction with confidence ≥ 0.90 (90%)
Severity MEDIUM	ATTACK prediction with confidence < 0.90 (90%)
True Positive	Real attack correctly classified as ATTACK
False Positive	Normal traffic incorrectly classified as ATTACK (false alarm)
False Negative	Real attack incorrectly classified as BENIGN (missed detection)
F1-Score	Harmonic mean of Precision and Recall. Balance metric for classification
Stratified split	Train/test split that preserves the class ratio from the original dataset